Multi-Lingual Text Detection in Visual Media with Font-Agnostic OCR

BITE497J – Project I

Submitted in partial fulfillment of the requirements for the degree of

Bachelor of Technology

in

Information Technology

by

RIPUN SETHIA
23BIT0421

YASH JOSHI
23BIT0137

Under the guidance of
Dr. Chemmalar Selvi G

School of Computer Science Engineering and Information Systems
VIT, Vellore



VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

November, 2026

DECLARATION

We hereby declare that the BITE497J – Project I thesis entitled **"Multi-Lingual Text Detection in Visual Media with Font-Agnostic OCR"** submitted by us, for the award of the degree of **Bachelor of Technology in Information Technology, School of Computer Science Engineering and Information Systems** to VIT is a record of bonafide work carried out by us under the supervision of **Dr. Chemmalar Selvi G, SCORE, VIT, Vellore.**

We further declare that the work reported in this thesis has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: Vellore
Date:

Ripun Sethia (23BIT0421)

Yash Joshi (23BIT0137)

Signature of the Candidates

CERTIFICATE

This is to certify that the BITE497J – Project I thesis entitled **"Multi-Lingual Text Detection in Visual Media with Font-Agnostic OCR"** submitted by Ripun Sethia (23BIT0421) and Yash Joshi (23BIT0137), SCORE, VIT, for the award of the degree of **Bachelor of Technology in Information Technology, School of Computer Science Engineering and Information Systems**, is a record of bonafide work carried out by them under my supervision during the period, 06.07.2026 to 03.11.2026, as per the VIT code of academic and research ethics.

The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university. The thesis fulfills the requirements and regulations of the University and in my opinion meets the necessary standards for submission.

Place: Vellore
Date:

Signature of the Guide

Internal Examiner **External Examiner**

Head of the Department
Department of Information Technology

ACKNOWLEDGEMENT

It is our pleasure to express with a deep sense of gratitude to our BITE497J – Project I guide **Dr. Chemmalar Selvi G**, School of Computer Science Engineering and Information Systems, Vellore Institute of Technology, Vellore, for her constant guidance and continual encouragement in our endeavour. Our association with her is not confined to academics only; it has been a great opportunity on our part to work with an intellectual and an expert in the field of **Computer Vision and Optical Character Recognition**.

We would like to express our heartfelt gratitude to Honorable Chancellor **Dr. G Viswanathan**; respected Vice Presidents **Dr. Sankar Viswanathan**, **Dr. Sekar Viswanathan**; Vice Chancellor **Dr. V. S. Kanchana Bhaaskaran**; Pro-Vice Chancellor **Dr. Partha Sharathi Mallick**; and Registrar **Dr. Jayabarathi T**.

Our whole-hearted thanks to Dean **Dr. Daphne Lopez**, School of Computer Science Engineering and Information Systems; Head, Department of Information Technology, **Dr. Arivuselvan K**; Information Technology Project Coordinator **Dr. Suganya P**; SCORE School Project Coordinator **Dr. Karthikeyan P**; and all faculty, staff and members working as limbs of our university for their continuous guidance throughout our course of study in unlimited ways.

It is indeed a pleasure to thank our parents and friends who persuaded and encouraged us to take up and complete this project successfully. Last, but not least, we express our gratitude and appreciation to all those who have helped us directly or indirectly towards the successful completion of the project.

Place: Vellore
Date:

Ripun Sethia
Yash Joshi

EXECUTIVE SUMMARY

This project builds an optical character recognition (OCR) system that reads English, Tamil and Hindi (Devanagari) text from images and identifies the script automatically, so the user does not have to choose a language. The current architecture (V2) runs one PaddleOCR detection-and-recognition engine per language and selects the script whose engine produces the most confident text inside its own Unicode block, with an override that prefers Tamil or Hindi once either holds at least 25% of the total in-script glyph mass. It is delivered as a command-line tool and a FastAPI web application with a Simple view, a Developer view and a Research tab, and is covered by 41 automated tests (20 production, 21 legacy), all passing.

The report also documents the earlier modular architecture (V1: EAST detection, geometric grouping, a printed/handwritten heuristic and TrOCR, later extended with separate Tamil and Hindi back-ends), the defects found in it, and the reasons for replacing it. Accuracy was measured on real third-party handwriting: 24.1% character error rate (CER) on English sentence lines, 47.7% on Hindi words and 71.1% on Tamil words, thirty images each. These figures use different units and datasets and are not a ranking of script difficulty. A failure mode named script leakage, in which the Tamil engine emits Latin letters, was found in 17 of 30 Tamil predictions.

Automatic script identification was evaluated on 150 real handwriting images and was correct on 77.3% (English 96%, Hindi 86%, Tamil 50%). The Tamil weakness traces to the Tamil recogniser, which emitted no Tamil characters in 21 of the 25 misidentified Tamil images. The first, plain rule scored 65.3% overall and 70.0% on its held-out split; the final rule added the override after failures were observed, with a hand-chosen threshold, so its held-out result is indicative and not a clean estimate.

CONTENTS

Declaration ii
Certificate iii
Acknowledgement iv
Executive Summary v
Contents vi
List of Figures vii
List of Tables vii
List of Abbreviations and Symbols viii
1 INTRODUCTION 1
1.1 Objective 1
1.2 Motivation 1
1.3 Background 1
1.4 Organisation of the Report 2
2 PROJECT DESCRIPTION AND GOALS 3
2.1 Problem Statement 3
2.2 Project Evolution Since Review-1 3
2.3 Goals and Contributions 3
2.4 Scope and Non-Goals 4
2.5 Literature Survey 4
2.6 Findings and Identified Gap 5
3 TECHNICAL SPECIFICATION 6
3.1 Functional Requirements 6
3.2 Non-Functional Requirements 6
3.3 Software Environment 6
3.4 Hardware Environment 6
3.5 Recognition Models 7
3.6 Datasets 7
4 DESIGN APPROACH AND DETAILS 8
4.1 Design Approach / Materials and Methods 8
4.2 Codes and Standards 14
4.3 Constraints, Alternatives and Trade-offs 14
5 LEGACY ARCHITECTURE AND MIGRATION 16
5.1 The V1 Pipeline 16
5.2 The Grouping Algorithm and Its Defect 16
5.3 Extending V1 to Tamil and Hindi 17
5.4 Why V1 Was Replaced 17
5.5 Preservation of V1 17
6 SCHEDULE, TASKS AND MILESTONES 18
6.1 Timeline 18
6.2 Task Breakdown 18
6.3 Corrections Made During Documentation 19
7 PROJECT DEMONSTRATION 20
7.1 Installation 20
7.2 Command-Line Usage 20
7.3 Sample Run on an English Examination Page 20
7.4 Web Demonstration 20
7.5 Testing 21
8 RESULT AND DISCUSSION 23
8.1 Evaluation Methodology 23
8.2 V1 Results 23
8.3 V2 Qualitative Results on the Sample Page 24
8.4 V2 Real-Data Character Error Rate 24
8.5 Script Leakage 25
8.6 Automatic Script Identification 26
8.7 Latency 27
8.8 Feature Comparison With Other OCR Options 27
8.9 Vendor-Reported Accuracy 28
8.10 Discussion 28
8.11 Limitations and Threats to Validity 29
9 SUMMARY 30
10 REFERENCES 31
APPENDIX A 32
A.1 Script Identification Module (recognition/script_detector.py) 32
A.2 Orchestration of Manual and Automatic Modes (main.py) 32
A.3 Reproducing the Evaluation and Running the Demo 32

LIST OF FIGURES

Fig. 1.1 Two-Stage OCR Pipeline With Detection and Recognition 2
Fig. 4.1 System Context Diagram 8
Fig. 4.2 Layered Component Architecture of V2 9
Fig. 4.3 Sequence Diagram of an OCR Request 10
Fig. 4.4 Flowchart of Automatic Script Identification (Argmax With Indic Override) 12
Fig. 4.5 Class and Module Diagram of V2 14
Fig. 5.1 V1 Pipeline After the Tamil and Hindi Extensions 16
Fig. 6.1 Project Milestones From the Git History 18
Fig. 7.1 Regions Returned by V2 on the Sample Examination Page 20
Fig. 7.2 Web Interface, Simple View: Auto-Detect Result on the Sample Page 21
Fig. 7.3 Web Interface, Developer View: Region Boxes, Evidence Bars, Crops and Confidence 21
Fig. 8.1 Tamil CER Against Box Padding in V1 (5 Synthetic Images) 24
Fig. 8.2 V2 Real-Data CER With V1 Synthetic Pilot Values (Units Differ Across Languages) 25
Fig. 8.3 Distribution of Per-Image CER for Each Language 25
Fig. 8.4 Latin Script Leakage: Frequency and Effect on CER 26

LIST OF TABLES

Table 1.1 Unicode Ranges of the Three Supported Scripts 2
Table 2.1 State of the Project at Review-1 and at Final Submission 3
Table 2.2 Contributions, Evidence and Strength of Each Claim 4
Table 2.3 Literature Survey 4
Table 3.1 Functional Requirements 6
Table 3.2 Pinned Software Dependencies 6
Table 3.3 Recognition Engines Used per Language 7
Table 3.4 Evaluation Data 7
Table 4.1 Components and Responsibilities 9
Table 4.2 Region Record 10
Table 4.3 Detection Record 11
Table 4.4 HTTP Endpoints 11
Table 4.5 Characters Counted for Each Script 12
Table 4.6 Design Decisions, Alternatives and Trade-offs 15
Table 5.1 Comparison of the V1 and V2 Architectures 17
Table 6.1 Milestones and Representative Commits 18
Table 6.2 Task Breakdown and Status 18
Table 7.1 Excerpt of the V2 Output for the Sample Page 20
Table 7.2 Automated Test Inventory 22
Table 8.1 Experiments 23
Table 8.2 Examples of Text Recovered by the Grouping Fix 23
Table 8.3 V2 Character Error Rate on Real Handwriting (30 Images per Language) 24
Table 8.4 Examples of Ground Truth, Prediction and Per-Sample CER 25
Table 8.5 Script Leakage in the Indic Engines 26
Table 8.6 Script Identification Accuracy of the Final Rule (Argmax With Indic Override), E7 26
Table 8.7 Script Identification Accuracy of the First Rule (Plain Argmax), E7 26
Table 8.8 Confusion Matrix of the Final Rule (All 150 Images; Rows Are the True Script, Columns the Prediction) 26
Table 8.9 Latency on the Development CPU (Warm Models) 27
Table 8.10 Feature Comparison With Other OCR Options (Features Only; No Accuracy Comparison Was Run) 28

LIST OF ABBREVIATIONS AND SYMBOLS

Abbreviations

API	Application Programming Interface
CER	Character Error Rate
CLI	Command-Line Interface
CNN	Convolutional Neural Network
CORS	Cross-Origin Resource Sharing
CPU	Central Processing Unit
CRAFT	Character Region Awareness for Text detection
CRNN	Convolutional Recurrent Neural Network
CTC	Connectionist Temporal Classification
CVIT	Centre for Visual Information Technology (IIIT Hyderabad)
DB	Differentiable Binarization
EAST	Efficient and Accurate Scene Text detector
GPU	Graphics Processing Unit
HTTP	Hypertext Transfer Protocol
IAM	IAM Handwriting Database (University of Bern)
IIIT	International Institute of Information Technology
JSON	JavaScript Object Notation
MKL-DNN	Math Kernel Library for Deep Neural Networks (now oneDNN)
NMS	Non-Maximum Suppression
OCR	Optical Character Recognition
OSD	Orientation and Script Detection (Tesseract)
PARSeq	Permuted Autoregressive Sequence model
PP-OCR	PaddlePaddle OCR model family
REST	Representational State Transfer
TrOCR	Transformer-based OCR
UI	User Interface
UTF-8	8-bit Unicode Transformation Format
V1 / V2	Previous (legacy_v1/) and current architecture
VLM	Vision-Language Model
WER	Word Error Rate
YAML	YAML Ain't Markup Language

Symbols and Notations

s	A script: english, tamil or hindi
S	Set of supported scripts {english, tamil, hindi}
$B(s)$	Character set (Unicode block) counted for script s
t_i	Recognized text of region i
c_i	Recognition confidence of region i, $0 \le c_i \le 1$
$M(s)$	Confidence-weighted in-script glyph mass of script s
$\hat{s}$	Script selected by automatic identification
share	Winner's fraction of total glyph mass
$d(p, g)$	Levenshtein distance between prediction p and ground truth g
$\text{len}(g)$	Length of ground truth g in Unicode code points
$n(t, s)$	Number of characters of text t that belong to $B(s)$
N	Number of evaluated images
θ	Indic override threshold (0.25)
$\bar{h}$	Average height of detected boxes (V1 grouping)
v_tol, h_gap	Vertical and horizontal grouping tolerances (V1)
σ	Standard deviation of grayscale pixel values (V1 classifier)

CHAPTER 1 — INTRODUCTION

Text embedded in images is one of the densest carriers of information in visual media. Examination scripts, signboards, forms, notes, screenshots and social-media graphics all encode meaning that a human reader can use immediately but that software cannot use until the text has been transcribed. Optical Character Recognition (OCR) is the family of techniques that performs this transcription. In a country such as India the same task has an additional dimension: a single office, classroom or street may present text in English, Hindi, Tamil and several other scripts, frequently handwritten, and the user of an OCR tool should not have to know in advance which script a given image contains.

At Review-1 this project presented a modular English-only prototype that detected text with the EAST scene-text detector, grouped word boxes into lines, classified each line as printed or handwritten, and routed it to a TrOCR recogniser. Since then the prototype has been audited and repaired, extended to Tamil and Hindi, evaluated on real handwriting, and finally re-architected around one PaddleOCR engine per language with automatic script identification. This report describes the final system, the legacy architecture it replaced, the evaluation, and the honest limits of what was achieved.

1.1 Objective

The primary objective of the project is to build a usable multilingual OCR tool that reads English, Tamil and Hindi text from images, removes the need for the user to choose the language, and reports its own accuracy on real data using a clearly defined metric. The specific objectives are listed below.

1. To identify and fix correctness defects in the Review-1 prototype and protect each fix with automated regression tests.
2. To support three languages, English (Latin script), Tamil and Hindi (Devanagari script), through a clean architecture in which adding a language does not require a new code path.
3. To measure recognition accuracy on real, third-party handwriting datasets using the character error rate (CER), and to report exactly what was measured and on what data.
4. To remove the need for the user to select a language by identifying the script of the image automatically.
5. To provide a presentable web application and a command-line interface, together with complete technical documentation.
6. To extract defensible engineering and evaluation findings from the work, while explicitly avoiding claims of algorithmic novelty that the evidence does not support.

1.2 Motivation

Most OCR tools ask the user to state the language of the document before recognition. This is a reasonable request for a translator working on a known document, but an unreasonable one for a teacher photographing a stack of mixed-language answer sheets or for an automated pipeline that receives images from many sources. A wrong choice silently produces garbage, and the user may not read the script well enough to notice.

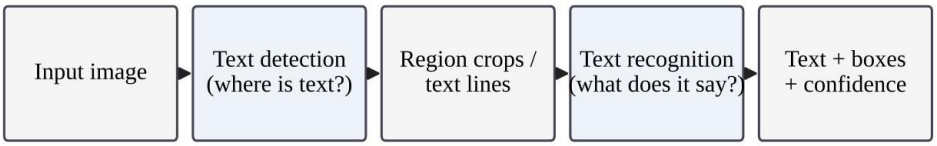
Open-source OCR for Indic scripts is also far less mature and far less evaluated than OCR for English. Vendors publish accuracy figures on their own test sets, usually for printed or synthetic text, and these figures rarely describe what a user will see on real handwriting. A tool that is honest about how accurate it really is, and that shows the user its confidence for every region it reads, is more useful than one that quotes a single optimistic benchmark.

Finally, the Review-1 prototype had a modular design whose components were each reasonable in isolation, and the project offered an opportunity to test that design against reality: extending it to two new scripts exposed failures that are instructive in their own right and that motivated the move to a unified engine.

1.3 Background

1.3.1 The OCR Pipeline

OCR is conventionally decomposed into two sub-problems, shown in Fig. 1.1. Text detection localises the regions of an image that contain text and returns bounding boxes or polygons; text recognition converts each localised region into a character sequence. Detectors such as EAST [2], CRAFT [4] and differentiable-binarization (DB) detectors [17] address the first problem; sequence recognisers such as CRNN [3], PARSeq [6] and TrOCR [5] address the second. Toolkits such as PaddleOCR [7], [8] package a detector and a recogniser together behind one call.



Script identification can sit before recognition (choose the model) or after it (judge the outputs) - this project does the latter.

Fig. 1.1 Two-Stage OCR Pipeline With Detection and Recognition

1.3.2 Characteristics of the Tamil and Devanagari Scripts

A script is a writing system; a language is what is written in it. Hindi, Marathi and Nepali share the Devanagari script, while Tamil has its own script. Both Indic scripts are abugidas: a consonant carries an inherent vowel that is changed by dependent vowel signs, and a virama sign suppresses the vowel so that consonants can join. In Devanagari, joined consonants form conjuncts and most letters hang from a horizontal headline called the shirorekha, which makes words visually connected. These properties affect OCR in two ways that matter for this report. First, a single visible character (a grapheme cluster) is frequently stored as several Unicode code points, so an error metric computed over code points penalises Indic text more heavily than English. Second, the connected appearance of Devanagari defeats detectors trained mainly on separated Latin words, as Chapter 5 shows for EAST.

Table 1.1 Unicode Ranges of the Three Supported Scripts

Script	Languages (examples)	Unicode range used in this project	Notable features
Latin	English	ASCII letters A–Z, a–z	Separate letters, word spaces, small alphabet
Tamil	Tamil	U+0B80 – U+0BFF	Abugida, virama (pulli), compound vowel signs
Devanagari	Hindi, Marathi, Nepali	U+0900 – U+097F	Abugida, conjuncts, headline (shirorekha)

1.3.3 Measuring OCR Accuracy

The standard measure for recognition quality is the character error rate: the Levenshtein edit distance [16] between the predicted and the reference text, divided by the length of the reference. A CER of zero is a perfect transcription; a CER above one is possible when the prediction is much longer than the reference. The exact definition used in this project is given in Section 8.1. Word error rate (WER) and exact-match accuracy are common complements; only exact match was computed here.

1.3.4 Script Identification

Script identification decides which writing system an image or region contains, and it is an established research problem with dedicated convolutional models [12] and surveys [13]. It can be performed before recognition, by a separate classifier that chooses the recogniser, or after recognition, by judging the outputs of several recognisers. This project uses the second approach because it needs no additional model, no training data and no extra download; Section 4.1.10 describes the method and its limits.

1.4 Organisation of the Report

Chapter 2 states the problem, traces the evolution of the project since Review-1, lists the contributions with an honest rating of each, and surveys related work. Chapter 3 gives the technical specification. Chapter 4 presents the design of the current system, including the automatic script identification method and the UML diagrams. Chapter 5 describes the legacy architecture and why it was replaced. Chapter 6 gives the schedule and milestones, Chapter 7 demonstrates the system, and Chapter 8 presents the evaluation methodology, results, comparison with other tools, discussion and threats to validity. Chapter 9 summarises the work and the future directions. Appendix A contains selected source code and reproduction commands.

CHAPTER 2 — PROJECT DESCRIPTION AND GOALS

2.1 Problem Statement

The problem addressed by this project has three parts. First, a usable multilingual OCR tool must read every supported script, which for this project means English, Tamil and Hindi. Second, it must not require the user to name the language of the image before recognition. Third, it must be honest about its accuracy on real input, and real handwriting is far harder than the clean printed or synthetic text on which OCR tools are usually demonstrated. A tool that satisfies the first two requirements but quotes a misleading accuracy figure does not solve the problem.

The Review-1 prototype did not satisfy any of the three. It recognised English only, it had no notion of script, and no quantitative evaluation had been carried out. Its modular structure was intended to make extension straightforward; testing that intention was itself part of the work.

2.2 Project Evolution Since Review-1

The project began in March 2026 as an English prototype that worked on one sample image. In August 2026 it was audited, and a grouping defect that silently clipped recognised text was found and fixed. The modular pipeline (V1) was then extended to Tamil and Hindi, which exposed script-dependent detector failures. In October 2026 the system was re-architected around PaddleOCR (V2), evaluated on real handwriting, given automatic script identification, and fitted with a redesigned web interface. Table 2.1 compares the state of the project at Review-1 with its final state.

Table 2.1 State of the Project at Review-1 and at Final Submission

Aspect	Review-1 (August 2026)	Final (this report)
Languages	English (printed, handwritten)	English, Tamil, Hindi
Language choice	Not applicable	Automatic by default; manual override
Detection	EAST via OpenCV DNN	PaddleOCR detector per language (PP-OCRv6 / PP-OCRv5 server)
Recognition	TrOCR large printed / handwritten	PaddleOCR recogniser per language
Grouping and classification	Heuristic grouping; grayscale-std classifier	Not needed: the engine returns line regions, sorted into reading order
Interfaces	Script reading a fixed input folder	CLI with flags; FastAPI web application with Simple, Developer and Research views
Evaluation	None (design targets only)	CER on 90 real handwriting images; script identification on 150; leakage analysis; latency
Tests	None	41 automated tests (20 production, 21 legacy)

2.3 Goals and Contributions

The contributions of the project are engineering and evaluation findings, not a new algorithm. Table 2.2 lists each contribution together with the evidence behind it and an assessment of how strong the claim is. The assessment is part of the contribution: several early conclusions were weakened or withdrawn when the evidence was examined closely, and the corrections are recorded in Section 6.3.

Table 2.2 Contributions, Evidence and Strength of Each Claim

#	Contribution	Evidence	Strength
C1	A grouping bug in the prototype silently clipped OCR input; fixing it recovered clipped text	19 of 32 recognised lines gained text, none lost; regression tests fail before and pass after the fix	Solid for that pipeline and image; qualitative
C2	In V1 the EAST detector failed differently per script: grouping bug (English), under-sized boxes (Tamil), zero boxes (Hindi)	English: C1; Tamil: 5 synthetic images; Hindi: 1 synthetic image at 2 thresholds	Suggestive; small and synthetic
C3	Symmetric box padding partly mitigates Tamil under-sizing (CER 46.3% to 25.9% at 20 px; worse at 30 px)	5 synthetic images	Suggestive (n = 5)
C4	One PaddleOCR engine per language removed the detection failures and simplified the system	16 coherent regions vs 32 fragments on the sample page; one code path	Solid as engineering; not a controlled accuracy comparison
C5	On real handwriting the engines still err a lot: English 24.1%, Hindi 47.7%, Tamil 71.1% CER	30 real images per language	Real signal, small n; not comparable across languages
C6	A clean synthetic pilot made accuracy look far better than real data did	V1 pilots vs V2 real-data runs	Evaluation-practice caution only
C7	Script leakage: the Tamil engine outputs Latin letters on 17/30 real images (CER 0.91 vs 0.46)	Per-sample analysis of C5 outputs	Solid for this sample; mechanism not isolated
C8	Automatic script identification from recogniser outputs (plain argmax, then argmax with an Indic override)	150 real images (Section 8.6): 77.3% final rule, 65.3% first rule	Real signal, small n; Tamil unreliable (50%); final rule's held-out figure is not a clean estimate; first-rule held-out 70.0%

The following are explicitly **not** claimed: a new OCR or script-identification algorithm; that V2 is more accurate than V1 on the same data, since V1 was never run on the real datasets; that the three CER values order the scripts by difficulty, since English was scored on long lines and the Indic languages on isolated words; that the system is more accurate than any other OCR tool, since no such comparison was run; and that the results generalise beyond the small samples used.

2.4 Scope and Non-Goals

The scope of the delivered system is single still images containing text in one of the three supported scripts, processed on a CPU in a single process. The following are outside the scope of the project:

- A state-of-the-art recogniser: the system wraps an existing engine rather than training one.
- A deployed, hardened service: the web application is a demonstration without authentication, rate limiting or server-side upload limits.
- Page-layout analysis, tables, or per-region reading of documents that mix scripts on one page.
- GPU optimisation, batching or asynchronous processing.
- A patentable invention: a patent route was investigated and dropped, as Section 2.5 explains.

2.5 Literature Survey

Table 2.3 summarises fifteen works that bear directly on the design and evaluation choices of the project. The survey of Review-1 covered scene-text detection and Transformer recognition in depth; this survey keeps the works that the final system still depends on and adds the PaddleOCR family, the handwriting datasets used for evaluation, and work on script identification.

Table 2.3 Literature Survey

S.No	Work	Merits	Demerits / relevance
1	Adapting Tesseract for multilingual OCR (Smith et al., 2009) [1]	Shows one open engine can be generalised to many scripts through script-specific data and processing	Built for clean scanned documents; each script needs its own data; handwriting not addressed
2	EAST scene text detector (Zhou et al., 2017) [2]	Single fully convolutional network predicting word quadrilaterals; fast; usable from OpenCV	Word-level boxes need regrouping; returned zero boxes on Devanagari in this project
3	CRNN (Shi, Bai & Yao, 2017) [3]	Convolutional features + recurrent modelling + CTC; lexicon-free; long-standing baseline	Assumes roughly horizontal text; separate detector still required

S.No	Work	Merits	Demerits / relevance
4	CRAFT (Baek et al., 2019) [4]	Character-region and affinity maps handle arbitrary-shaped text	Used inside EasyOCR and ocr_tamil in V1; heavier than box detectors
5	TrOCR (Li et al., 2023) [5]	Pure Transformer recogniser from pretrained vision and language models; strong printed and handwritten results	English-centric checkpoints; large and slow on CPU (about 4 s per region in V1)
6	PARSeq (Bautista & Atienza, 2022) [6]	Permuted autoregressive training unifies context-free and context-aware decoding	Recogniser only; used through ocr_tamil in V1
7	PP-OCR (Du et al., 2020) [7]	Practical lightweight detector and recogniser with many engineering refinements	Accuracy depends on per-language models of varying maturity
8	PaddleOCR 3.0 technical report (Cui et al., 2025) [8]	Unified toolkit with multilingual recognition and document pipelines	Vendor-reported accuracy on vendor test sets; did not transfer to real handwriting here
9	PP-OCRv6 (Zhang et al., 2026) [9]	Reports small models rivalling billion-parameter VLMs on OCR tasks	English-only generation in this project; Indic languages still use PP-OCRv5
10	IAM database (Marti & Bunke, 2002) [10]	Standard English handwritten sentence-line benchmark	Lines, not words; research-use licence terms apply
11	IIIT-INDIC-HW-WORDS (Gongidi & Jawahar, 2021) [11]	Large Indic handwritten word dataset (872K words, 135 writers per its project page)	Isolated words only; licence of the mirror used here not stated
12	Script identification in the wild (Shi, Bai & Yao, 2016) [12]	Discriminative CNN for script identification of scene text	Needs training data and an extra model; motivates the training-free approach here
13	Script identification survey (Ubul et al., 2017) [13]	Establishes script identification as a mature research area	Confirms that script identification is not a novelty claim for this project
14	Synthetic data for text localisation (Gupta et al., 2016) [14]	Synthetic rendering yields large annotated data at low cost	Domain gap to real capture; consistent with the synthetic-vs-real gap seen in this project
15	CC-OCR benchmark (Yang et al., 2025) [15]	Evaluates large multimodal models on multilingual and scene OCR	Confirms multilingual OCR outside English and Chinese remains unsolved

Related searches during development also found that confidence-routed modular pipelines are common in production, that cascading error in modular pipelines is a named problem addressed by end-to-end models, and that low-resource script OCR is an active and less saturated research direction. Patents already exist on clustering and consolidating OCR boxes, and directions such as document-forgery detection and automated answer-sheet grading were judged too crowded to offer a contribution; the patent route was therefore dropped. These findings came from search-result summaries rather than full papers and are treated as pointers, not as a literature review.

2.6 Findings and Identified Gap

Three findings from the literature shaped the project. First, detection and recognition fail in different ways and are often provided by different models, so a detector that suits one script may not suit another. Second, mature toolkits now provide per-language detection and recognition models behind one interface, which makes it unnecessary to assemble a patchwork of back-ends. Third, published accuracy is almost always measured on the publisher's own data. The gap addressed is therefore practical rather than algorithmic: a small, tested, honestly evaluated tool for three Indian-relevant scripts that chooses the script automatically and states plainly how well it performs on real handwriting.

CHAPTER 3 — TECHNICAL SPECIFICATION

3.1 Functional Requirements

Table 3.1 lists the functional requirements of the delivered system. All of them are implemented and exercised by the command-line interface, the web application or the automated tests.

Table 3.1 Functional Requirements

ID	Requirement
FR1	Accept a still image (PNG, JPEG or BMP) from the command line or through an HTTP upload.
FR2	Detect and recognise text regions in English, Tamil or Hindi using one engine per language.
FR3	In automatic mode, run all three engines and select the script with the largest confidence-weighted in-script glyph mass, preferring Tamil or Hindi once either holds at least 25% of the total mass.
FR4	In manual mode, run only the engine of the language chosen by the user.
FR5	Fall back to English and set a fallback flag when no engine produces any in-script text.
FR6	Return, for each region, its index (reading order), bounding box, text, confidence score, engine label and script.
FR7	Save a cropped image of every region and write ocr_results.json, ocr_results.txt and ocr_detection.json.
FR8	Expose health, research, sample, OCR and crop endpoints over HTTP and serve the web interface.
FR9	Provide a Simple view (combined text with Copy, Download and Share), a Developer view (regions, crops, confidence, evidence) and a Research view.
FR10	Allow configuration through config.yaml with command-line flags taking precedence.

3.2 Non-Functional Requirements

- **Responsiveness:** all three models are loaded once at server start-up so that the first request is not delayed by model loading.
- **Testability:** the script-identification logic consists of pure functions, and recogniser construction is lazy, so tests run without models or network access.
- **Security of file handling:** the client filename is never used in a path; only a whitelisted extension is kept, and crop URLs are validated against strict regular expressions.
- **Portability:** the system runs on CPU only and has been developed on Windows 11; nothing in the code is platform-specific apart from documented environment notes.
- **Transparency:** every recognised string is traceable to a saved crop and a confidence score, and the web interface displays measured accuracy for each language.
- **Maintainability:** recognition, script identification, registry, configuration and interfaces live in separate modules with small interfaces.

3.3 Software Environment

The system is implemented in Python 3.11.4. All dependencies are pinned so that results are reproducible; Table 3.2 lists them by requirements file.

Table 3.2 Pinned Software Dependencies

Requirements file	Packages and versions	Purpose
requirements.txt	numpy 2.3.5, opencv-python 4.13.0.92, pillow 12.1.1, PyYAML 6.0.2, paddlepaddle 3.3.1, paddleocr 3.7.0 (pulls paddlex 3.7.2)	Production pipeline
requirements-webapp.txt	fastapi 0.143.0, uvicorn 0.54.0, python-multipart 0.0.32	Web API and server
requirements-dev.txt	pytest 9.1.1, datasets 5.1.0	Tests and dataset download
requirements-legacy.txt	PyTorch, transformers, ocr_tamil, EasyOCR	V1 architecture only

3.4 Hardware Environment

Development and all measurements were carried out on a laptop with an AMD Ryzen 5 5625U processor and 15.3 GB of RAM running Windows 11, without a GPU. All latency figures in Chapter 8 refer to this machine. PaddlePaddle's MKL-DNN (oneDNN) acceleration is disabled because it crashed on this CPU (Section 4.3), so other machines may run faster with it enabled.

3.5 Recognition Models

Which model family each language receives is decided inside PaddleOCR 3.7.0. The selection in Table 3.3 was verified from the PaddleOCR source and from its model-creation logs, and the engine labels attached to every region are taken from the project's own label table. If the PaddleOCR version pin changes, these labels must be re-verified.

Table 3.3 Recognition Engines Used per Language

Language	Detection model	Recognition model	Engine label in output
English	PP-OCRv6_medium_det	PP-OCRv6_medium_rec	PaddleOCR PP-OCRv6 (en)
Tamil	PP-OCRv5_server_det	ta_PP-OCRv5_mobile_rec	PaddleOCR PP-OCRv5 (ta)
Hindi	PP-OCRv5_server_det	devanagari_PP-OCRv5_mobile_rec	PaddleOCR PP-OCRv5 (hi)

3.6 Datasets

Accuracy was measured on real handwriting from third-party datasets obtained through Hugging Face mirrors. None of the data is redistributed with the project, and the licence status of every dataset is recorded as requiring review. The samples are the first rows of each dataset stream in order, not a random sample. Table 3.4 lists the data.

Table 3.4 Evaluation Data

Data	Source	Unit	Used
english_real	Teklia/IAM-line, test split, mirror of the IAM Handwriting Database [10], [24]	Handwritten sentence lines, mean 60.9 characters	First 30
tamil_real	c3rl/IIIT-INDIC-HW-WORDS-Tamil, train split (CVIT, IIIT Hyderabad) [11], [25]	Handwritten words, mean 9.6 characters	First 30
hindi_real	c3rl/IIIT-INDIC-HW-WORDS-Hindi, train split [25]	Handwritten words, mean 6.4 characters	First 30
script_id	The same three sources	As above	First 50 each (150)
tamil_pilot, hindi_pilot	Self-generated with PIL and the Nirmala font on a white background	Synthetic phrases	5 each
sample.png	Pre-existing English examination page in the repository	One page	1

The ICDAR 2021 paper describing IIIT-INDIC-HW-WORDS lists eight scripts and does not include Devanagari; the same group released Devanagari handwriting earlier as IIIT-HW-Dev [18]. The exact provenance of the Hindi mirror was not verified and is recorded as a limitation.

CHAPTER 4 — DESIGN APPROACH AND DETAILS

4.1 Design Approach / Materials and Methods

The current architecture, referred to as V2, follows one principle: each supported language is served by one complete detection-and-recognition engine, and everything else in the system is orchestration around those engines. This replaces the V1 design, in which detection, grouping, classification and recognition were separate stages assembled differently for each script. The sections below describe the system from the outside in: its context, its components, the recogniser wrapper, the request workflow, the data it produces, its interfaces, and the script identification method.

4.1.1 System Context

Fig. 4.1 shows the system in its environment. A user reaches the same pipeline either through the web interface, which posts the image to the FastAPI application, or through the command line. Both call run_pipeline(), which uses one or three PaddleOcrRecognizer engines and, in automatic mode, the script detector. The system runs as one process with no database, queue, authentication or external service at run time; the only network access is the one-time download of model weights into the PaddleX model cache.

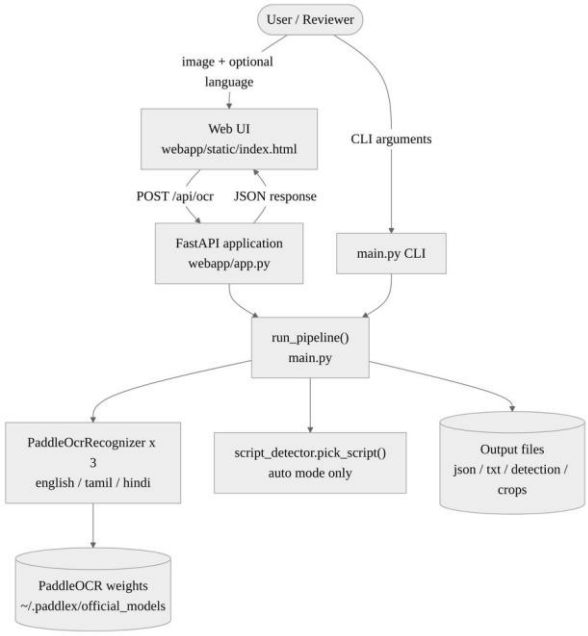


Fig. 4.1 System Context Diagram

4.1.2 Layered Component Architecture

Fig. 4.2 arranges the components into five layers and Table 4.1 states the responsibility of each. The presentation layer contains the single-file web interface and the command-line interface. The entry-point layer turns their inputs into calls of the orchestration layer, which reads the image, decides between manual and automatic mode, crops the regions and writes the outputs. The recognition layer contains the PaddleOCR wrapper, the script detector and the engine-label registry. Configuration and the external PaddleOCR dependency sit at the bottom.

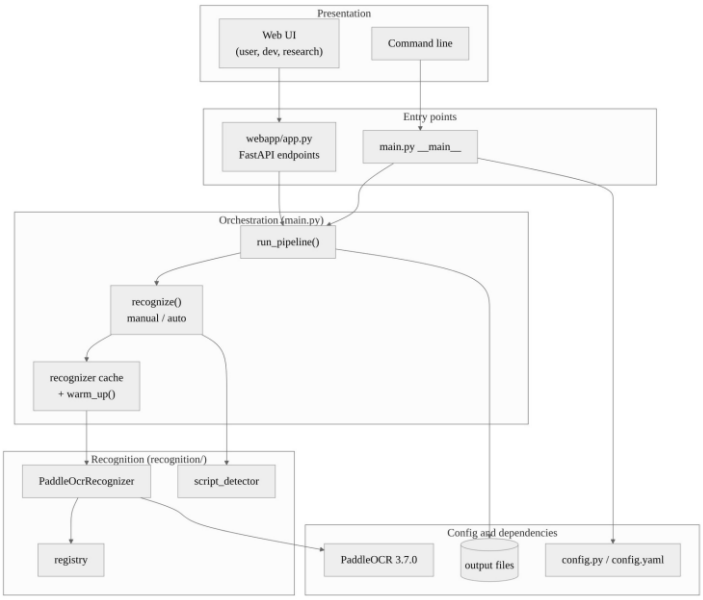


Fig. 4.2 Layered Component Architecture of V2

Table 4.1 Components and Responsibilities

Layer	File	Responsibility
Entry points	main.py (CLI), webapp/app.py (HTTP)	Parse input, call run_pipeline, return results
Orchestration	main.py: recognize(), run_pipeline()	Manual vs automatic mode, crops, output files
Recognition	recognition/paddle_recognizer.py	One PaddleOcrRecognizer(script) wraps one PaddleOCR detection + recognition pipeline and sorts regions into reading order
Script identification	recognition/script_detector.py	Pure functions: in-script glyph mass, argmax and Indic override
Registry	recognition/registry.py	Maps a script name to its engine label
Configuration	config.py, config.yaml	script, paths and logging settings merged over defaults
Frontend	webapp/static/index.html	Single-file interface: Auto-detect and manual language choice, Simple and Developer views, evidence bars, Research tab

4.1.3 The PaddleOcrRecognizer Wrapper

PaddleOcrRecognizer is the only class that touches PaddleOCR. One instance is created per script, and the constructor only validates the script name and stores the PaddleOCR language code (en, ta or hi); an unsupported name raises a ValueError. The PaddleOCR object itself is created lazily on the first call to warm_up() or detect_and_recognize(), which keeps construction free in tests. It is created with document orientation classification, document unwarping and text-line orientation disabled, which avoids loading three additional preprocessing models that the use case does not need, and with MKL-DNN disabled to avoid a crash described in Section 4.3.

detect_and_recognize(image_path) calls PaddleOCR's predict() on the image and converts each recognised line into a dictionary with an index, an axis-aligned bounding box [x1, y1, x2, y2] in original image pixels, the recognised text, the confidence score rounded to four decimals, and the engine label. The regions are then passed through sort_reading_order(), which groups regions into lines when their vertical centres lie within half a box height of each other, orders each line from left to right, and re-numbers the index. This corrects cases in which PaddleOCR returned a word before its left neighbour on the same line. Because PaddleOCR performs detection and recognition internally, the wrapper is always self-detecting: one call per image, with no separate grouping, clamping or classification stage.

4.1.4 Process-Lifetime Cache and Warm-Up

Loading a PaddleOCR pipeline takes seconds, so main.py keeps a process-lifetime dictionary _RECOGNIZER_CACHE that maps each script to its recogniser. warm_up() constructs and loads all three. The FastAPI lifespan hook calls

warm_up() at server start, so that a live audience never waits for a cold model load, and the /api/health endpoint reports which languages are warm; the web interface shows this as a status pill.

4.1.5 Request Workflow

Fig. 4.3 traces one request. The browser uploads an image and a script choice; the server validates the choice and stores the upload under a random twelve-hexadecimal-digit run identifier. run_pipeline() then runs either the single chosen engine or all three. In automatic mode the regions of the winning script are reused as the result, so no second recognition pass is needed. Each bounding box is clamped to the image and cropped, the output files are written, and the JSON response returns the regions, the detection information and the elapsed time. In the Developer view the browser then fetches the crops as thumbnails.

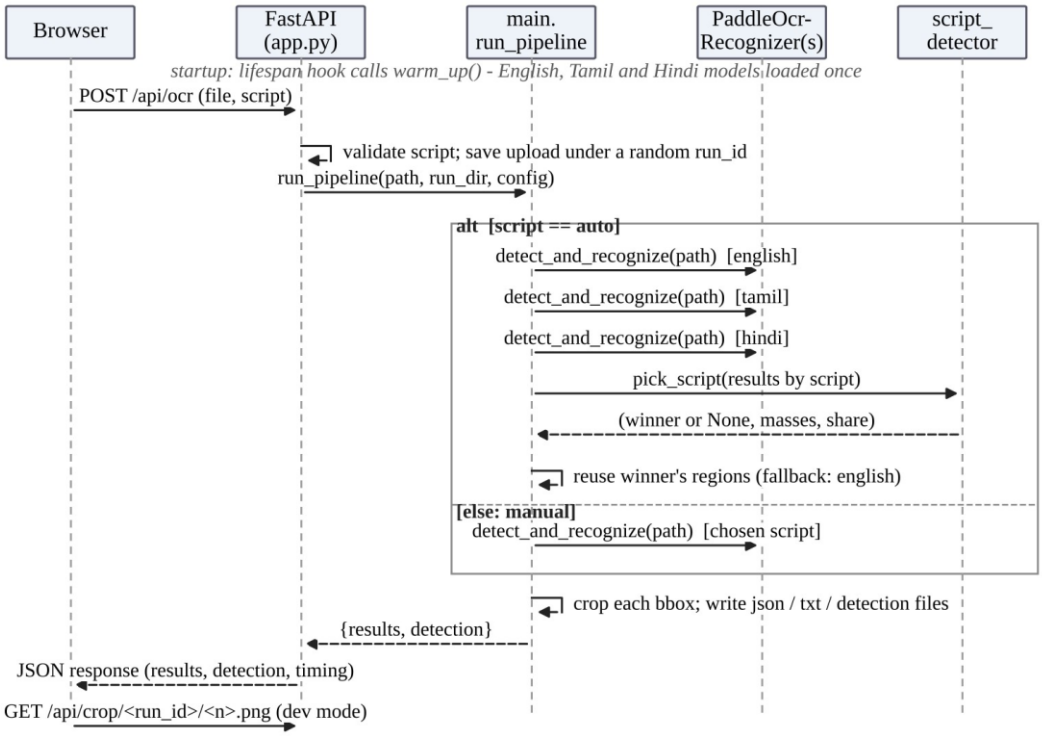


Fig. 4.3 Sequence Diagram of an OCR Request

4.1.6 Data Model

The result of a run is a list of region records, written to ocr_results.json and returned by the API, together with one detection record written to ocr_detection.json. Tables 4.2 and 4.3 define the two records.

Table 4.2 Region Record

Field	Type	Meaning
index	int	Reading-order index within the image, assigned after sorting
bbox	[x1, y1, x2, y2] ints	Axis-aligned box in original image pixels
text	string	Recognised text of the region
score	float 0–1	PaddleOCR recognition confidence, four decimals
route	string	Label of the engine that produced the region
script	string	Script the region was read as: english, tamil or hindi
crop_url	string	API only: /api/crop/{run_id}/{index}.png

Table 4.3 Detection Record

Field	Meaning
mode	auto or manual
script	Script finally used
fallback	true if automatic mode found no in-script text in any engine and defaulted to English
share	Winner's fraction of the total glyph mass (automatic mode only; null in manual mode). It is not a confidence: its median over the 150 evaluated images was 1.0
masses	Per-script confidence-weighted glyph mass (automatic mode only; null in manual mode)

4.1.7 HTTP API

The FastAPI application exposes the endpoints in Table 4.4. An unsupported script value or an unreadable image returns HTTP 400, and any other pipeline exception returns HTTP 500 with the message “Pipeline error”. The OCR response contains run_id, requested_script, script, detection, results, elapsed_seconds and was_warm. Crop requests are accepted only when the run identifier matches [a-f0-9]{12} and the filename matches a number followed by .png; a path-traversal attempt was confirmed to be rejected. CORS is open to all origins and there is no authentication, rate limiting or server-side upload-size limit, which is acceptable for a local demonstration and is listed as a limitation.

Table 4.4 HTTP Endpoints

Method and path	Purpose
GET /api/health	Returns status and warm-up state of english, tamil and hindi
GET /api/research	Static research-findings JSON used by the Research tab
GET /api/sample	The fixed sample page, used by the “Try sample” button
POST /api/ocr	Multipart form: file (image), script (auto by default, english, tamil, hindi)
GET /api/crop/{run_id}/{n}.png	Cropped region image; invalid identifiers return HTTP 400
GET /	Static frontend

4.1.8 Configuration and Command-Line Interface

config.yaml holds three settings: script (auto, english, tamil or hindi), the input and output paths, and the logging level. config.py merges the file over built-in defaults, so a missing file or a missing key falls back safely. The command-line flags --config, --input, --output-dir and --script override the file. In automatic mode the CLI logs the detected script and its share of the glyph mass.

4.1.9 Web Interface Design

The web interface is a single HTML file of about 814 lines with no build step, no external fonts or scripts, light and dark themes, and a layout verified down to a 390 px phone width. An image is supplied by drag-and-drop, by clicking, by pasting, or with the “Try sample” button. Auto-detect is the default language choice, alongside English, Tamil and Hindi, each with a short note that includes its measured accuracy. A status pill shows whether the models are warm, a running request shows a timer and a Cancel button, and failures are shown as friendly error states with a retry.

The **Simple view** joins the recognised regions into one text card with Copy, Download .txt and Share buttons (Share uses the Web Share API where available and copies otherwise) and underlines low-confidence lines. After an automatic run it shows a “Detected script” chip, a warning when no readable text was found, and “Read as <language>” buttons that re-run the image in another language. The **Developer view** draws the region boxes over the image with hover linking to a table of regions, shows per-script evidence bars, crop thumbnails, engine labels and confidence pills (green at 0.85 and above, amber at 0.60 and above, red below), and exposes the raw JSON. A **Research tab** renders the findings of Chapter 8, including what the project does not claim. The interface does not show the winner's share as a confidence, because it saturates at 1.0. Screenshots appear in Chapter 7.

4.1.10 Automatic Script Identification

The method rests on one observation: a recogniser shown text in a script it was not built for mostly emits low-confidence garbage or characters from another script. Therefore every engine reads the image, and the engine whose output contains the most confident text inside its own Unicode block wins. No model is trained or downloaded for this purpose; the evidence comes from the recognisers themselves. Table 4.5 defines the character set counted for each script. Digits, spaces and punctuation are ignored because they are shared by all three scripts.

Table 4.5 Characters Counted for Each Script

Script s	Character set B(s)
tamil	Tamil block, U+0B80 to U+0BFF
hindi	Devanagari block, U+0900 to U+097F
english	ASCII letters A–Z and a–z

For a script s whose engine returned regions with texts t_i and confidences c_i, the glyph mass is defined by Eqn. 4.1, where n(t, s) is the number of characters of the text t that belong to B(s). The plain winner is given by Eqn. 4.2. The Indic override of Eqn. 4.3 was added after the first rule was seen to fail: the English engine turns Indic text into Latin garbage that inflates the English mass, so that plain argmax chose English on a Hindi poster containing English words. With b the larger of M(tamil) and M(hindi), the override selects b whenever b holds at least θ = 0.25 of the total mass, and otherwise keeps the plain winner. The value 0.25 was chosen by hand and was not tuned. The reported share of the winner is given by Eqn. 4.4.

$$M(s) = \Sigma_i \, c_i \cdot n(t_i, s) \tag{4.1}$$

$$\hat{s}_0 = \operatorname{argmax}_{s \in S} M(s) \tag{4.2}$$

$$\hat{s} = b \;\; \text{if} \;\; M(b) / \Sigma_s M(s) \geq \theta, \;\; \text{else} \;\; \hat{s}_0 \tag{4.3}$$

$$share = M(\hat{s}) / \Sigma_{s \in S} M(s) \tag{4.4}$$

In Eqn. 4.1 the factor c_i weights each character by the engine's own confidence, so a confidently read word counts for more than a hesitant one. If the total mass in the denominator of Eqn. 4.4 is zero, no engine produced any in-script character; the pipeline then uses English and sets fallback to true. The complete procedure is shown in Fig. 4.4 and as pseudocode below.

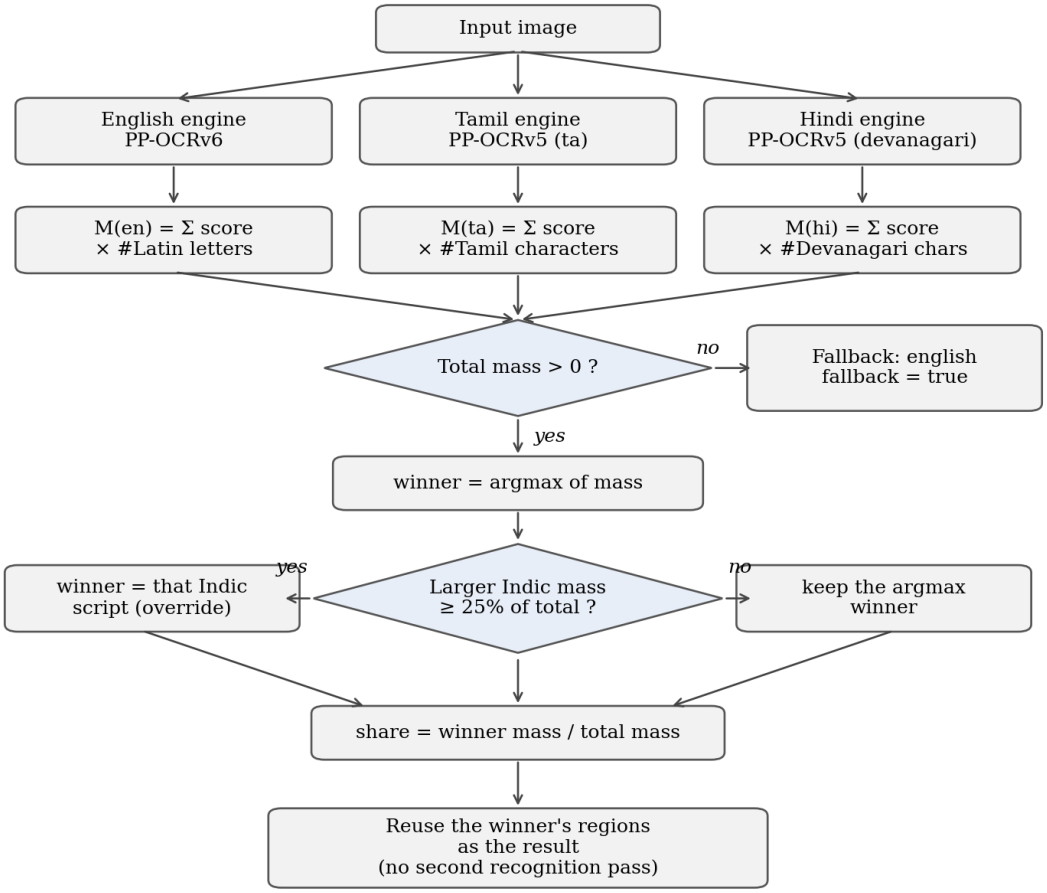


Fig. 4.4 Flowchart of Automatic Script Identification (Argmax With Indic Override)

```
for each script s in {english, tamil, hindi}:
    regions[s] = engine[s].detect_and_recognize(image)
    M[s] = sum over regions r of r.score * count(ch in r.text if ch in B(s))
total = M[english] + M[tamil] + M[hindi]
if total == 0: winner = english, fallback = true
else:
    winner = argmax(M)
    b = the larger of M[tamil], M[hindi]
    if M[b] / total >= 0.25: winner = b          # Indic override
    share = M[winner] / total
return regions[winner]                           # reused: no second recognition pass
```

An illustrative calculation follows. The values are constructed to show the arithmetic and are not measured outputs. Suppose a Hindi word image is read by all three engines: the Hindi engine returns a four-character Devanagari word with confidence 0.90 (mass 4 × 0.90 = 3.60), the Tamil engine returns three Latin letters with confidence 0.40 (no Tamil characters, mass 0), and the English engine returns two Latin letters with confidence 0.50 (mass 1.00). Hindi wins with share 3.60 / 4.60 = 0.78, and plain argmax and the override agree here. Had the English engine instead returned twelve Latin letters at confidence 0.90 (mass 10.8), the Hindi share would be 3.60 / 14.40 = 0.25, and the override would still select Hindi where plain argmax would select English.

The method has clear limits. When an Indic engine fails on hard handwriting it often emits Latin letters (Section 8.5); those letters add nothing to its own mass, so a failing Tamil engine leaves English to win, and the override cannot help when the Indic engine produced no Indic characters at all. The override can in principle route an English image to an Indic script; this did not happen on the 50 English images evaluated. Automatic mode costs about three times the compute of manual mode, a mixed-script document receives a single script, and languages sharing a script are not distinguished, so Marathi or Nepali text would be reported as hindi.

4.1.11 Class and Module Diagram

The class and module diagram in Fig. 4.5 reflects the code (it predates two small additions: the sort_reading_order function in the recogniser module and the sample endpoint). PaddleOcrRecognizer is the only class; main, script_detector, registry, config and the web application are Python modules and are shown with the module stereotype together with their functions and module-level state. main caches between zero and three recognisers, uses script_detector only in automatic mode, and is called by the web application. The two actors of the system are the User, who uploads an image, chooses a language or automatic detection, reads, copies or shares the text and may inspect regions or read the research findings, and the Developer or Reviewer, who additionally runs the command-line interface, checks server health and runs the tests and evaluation scripts; the system has no administrative or authentication use cases.

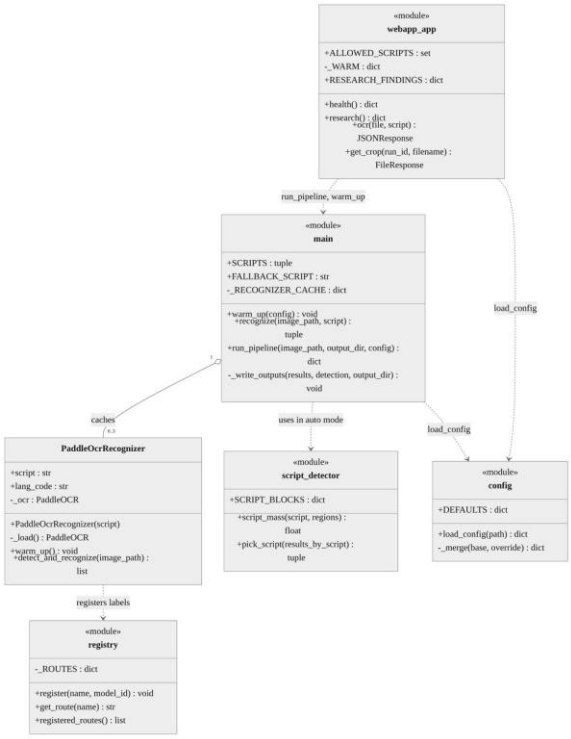


Fig. 4.5 Class and Module Diagram of V2

4.2 Codes and Standards

- **Unicode [19]:** script identification is defined over Unicode blocks (Tamil U+0B80–U+0BFF, Devanagari U+0900–U+097F), and all text is stored as Unicode strings.
- **UTF-8 (RFC 3629):** every output file is written in UTF-8, and standard output is reconfigured to UTF-8 so that Tamil and Devanagari can be logged on Windows consoles.
- **JSON (RFC 8259):** results, detection information and API responses use JSON, written with non-ASCII characters preserved.
- **HTTP and multipart/form-data (RFC 7578):** the OCR endpoint accepts an image and a script field as a multipart form.
- **Python packaging:** dependencies are pinned to exact versions in four requirements files, and tests use pytest.
- **Licences:** PaddleOCR and PaddlePaddle are Apache 2.0; V1 used TrOCR (MIT), ocr_tamil (MIT) and EasyOCR (Apache 2.0). The datasets and the EAST weights are recorded as requiring licence review.

4.3 Constraints, Alternatives and Trade-offs

Table 4.6 records the main design decisions, the alternatives that were considered, and the reason for each choice. Two environment constraints also shaped the implementation. With default settings, PaddlePaddle 3.3.1 raised NotImplementedError: ConvertPirAttribute2RuntimeAttribute not support during detection on the development CPU; disabling MKL-DNN avoids it. On Windows, pip install fails with an access-denied error on cv2.pyd while the server is running, so the server must be stopped before dependencies are changed.

Table 4.6 Design Decisions, Alternatives and Trade-offs

Decision	Alternatives considered	Choice and reason	Cost accepted
Engine architecture	Keep V1 per-language patchwork; end-to-end VLM OCR	One PaddleOCR engine per language: removes detector failures, one code path	Depends on PaddleOCR's internal model choice
Script identification	Trained script classifier; ask the user	Judge recogniser outputs by in-block glyph mass plus an Indic override: no extra model or data	About 3x compute; one script per image; hand-chosen threshold
Fallback when no evidence	Return empty result; raise an error	English, flagged with fallback = true	Caller must check the flag
Model loading	Load per request	Warm-up at start-up and process cache	Slower start; more memory
MKL-DNN	Leave enabled	Disabled to avoid a crash on the development CPU	Possibly slower on other CPUs
Legacy code	Delete V1	Preserve in legacy_v1/ with its own tests	Heavier repository
Error metric	Grapheme-level CER, WER	Code-point CER, identical code for all languages	Over-penalises Indic text
Frontend	Framework with a build step	One dependency-free HTML file served by FastAPI	Limited code structure

CHAPTER 5 — LEGACY ARCHITECTURE AND MIGRATION

The V1 architecture is preserved in the legacy_v1/ directory, runnable and tested, so that the comparison with V2 remains reproducible. This chapter describes it, the defects found in it, and the reasons it was replaced.

5.1 The V1 Pipeline

Fig. 5.1 shows V1 after its extension to three languages. For English and Tamil the image passed through the EAST detector, executed from a frozen TensorFlow graph by the OpenCV DNN module, whose score and geometry maps were decoded with a minimum confidence of 0.3 and non-maximum suppression with an overlap threshold of 0.3. Word boxes were grouped into lines, padded and clamped, and cropped. English crops were classified as printed or handwritten by a grayscale standard-deviation threshold of 70, which was never validated, and recognised with the large printed or large handwritten TrOCR checkpoint. Tamil crops were recognised with ocr_tamil (CRAFT and PARSeq). Hindi bypassed EAST entirely and used EasyOCR, which performs its own detection.

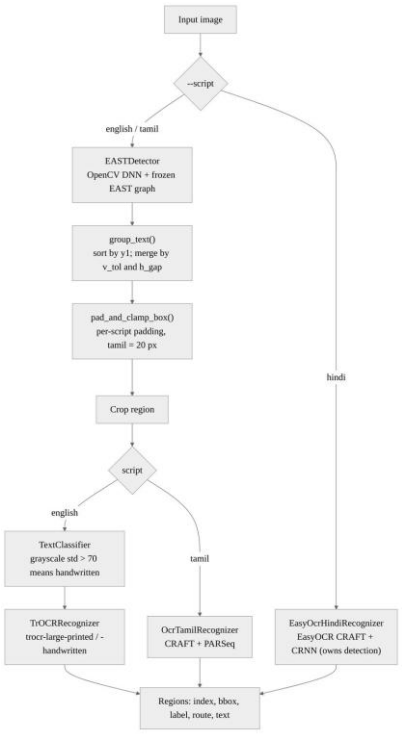


Fig. 5.1 V1 Pipeline After the Tamil and Hindi Extensions

5.2 The Grouping Algorithm and Its Defect

The grouping step sorted boxes by their top coordinate and merged neighbours that were vertically aligned and horizontally close. With $\bar{h}$ the average box height, the tolerances were given by Eqn. 5.1 and Eqn. 5.2.

$$v_tol = 0.5 \cdot \bar{h} \tag{5.1}$$

$$h_gap = 1.5 \cdot \bar{h} \tag{5.2}$$

A box joined the current group when the difference between its top coordinate and the group's top coordinate was below v_tol and the horizontal gap between its left edge and the group's right edge was below h_gap. The audit found that the pre-fix code grew only the right and bottom edges of the group with a maximum, while the left and top edges stayed pinned to the first box. Any later box that drifted left or up was therefore absorbed into the group without the crop rectangle expanding to include it, and the crop passed to TrOCR silently lost text. The fix, shown below, takes the minimum of the left and top edges as well and returns the groups in (y1, x1) reading order.

```
if abs(y1 - current[1]) < v_tol and (x1 - current[2]) < h_gap:
    current[0] = min(current[0], x1)   # added by the fix
    current[1] = min(current[1], y1)   # added by the fix
    current[2] = max(current[2], x2)
    current[3] = max(current[3], y2)
```

On the sample examination page 19 of 32 recognised lines gained text and none lost any; examples appear in Table 8.2. Eight regression and adversarial tests, including left-drift cases that fail on the pre-fix code, protect the fix.

5.3 Extending V1 to Tamil and Hindi

Extending V1 exposed a different detector failure for each new script. For Tamil, EAST produced boxes, but they were too small and cut off the edges of words: on five synthetic images every output was truncated, for example வணக்கம் was read as வணக்க், giving a CER of 46.3% with no exact matches. Symmetric padding of the boxes partly mitigated the problem; a padding of 20 pixels reduced the CER to 25.9% through the real pipeline, while 30 pixels made it worse again (Section 8.2). For Hindi, EAST returned no boxes at all on a synthetic Devanagari image, at a confidence threshold of 0.3 and again at 0.05, although the image contained 1,143 non-white pixels of text. EasyOCR's own detector read the same image correctly. Supporting Hindi therefore required a second recogniser contract, a “self-detecting” recogniser that takes the whole image instead of a crop.

5.4 Why V1 Was Replaced

By the time Hindi worked, V1 consisted of three back-ends, two recogniser contracts and two per-language geometric patches, and its English path was slow because TrOCR-large took about four seconds per region. Table 5.1 compares the two architectures on the properties that drove the decision. V2 is not claimed to be more accurate than V1 on identical data, because V1 was never run on the real datasets.

Table 5.1 Comparison of the V1 and V2 Architectures

Property	V1 (legacy_v1/)	V2 (current)
Back-ends	TrOCR, ocr_tamil, EasyOCR	PaddleOCR only
Recogniser contracts	Two (crop-based and self-detecting)	One (self-detecting)
Per-script patches	Grouping fix, Tamil padding, Hindi detector swap	None
Devanagari detection	EAST: zero boxes	Handled by PaddleOCR detector
Sample page result	32 fragments	16 coherent regions
Sample page latency (CPU)	126.8 s	14.5 s
Script identification	None (user chooses)	Automatic by default
Heavy dependencies	PyTorch, transformers, EAST weights	PaddlePaddle

5.5 Preservation of V1

V1 has its own registry copy, so importing both generations in one process cannot clash, its own configuration file, its own 21 tests and its own requirements file. It is run from the repository root with python -m legacy_v1.main --script english, after installing the legacy requirements and placing the EAST weights, which are not committed, in the models folder.

CHAPTER 6 — SCHEDULE, TASKS AND MILESTONES

6.1 Timeline

The repository history contains 52 commits: 13 on 26–27 March, 31 on 26–27 August and 8 on 9 October 2026. Fig. 6.1 places the main milestones on a timeline and Table 6.1 lists them with representative commits. Commit dates reflect when work was committed, which for the final phase was a single day.

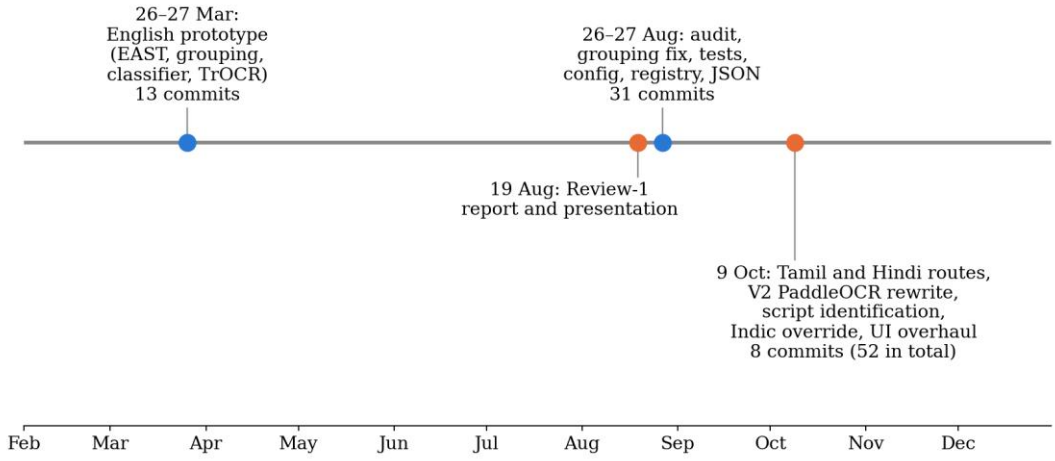


Fig. 6.1 Project Milestones From the Git History

Table 6.1 Milestones and Representative Commits

Date	Milestone	Commits (examples)
26–27 Mar 2026	Prototype: EAST detection, grouping, classifier, TrOCR; Chinese TrOCR route removed; README	7f11b45, a2d69dd, 38299d6
19 Aug 2026	Review-1 report and presentation	–
26 Aug 2026	Audit of the prototype; EAST missing-image handling	42e599e, 58b5b43
27 Aug 2026	Grouping fix and tests; CLI and UTF-8 logging; pinned dependencies; configuration; registry; box clamping; JSON output	e7ff323, 6a1cbc8, 57e4d62, 4205eb2, 5b8aca8, 7636214
9 Oct 2026	Tamil route (ocr_tamil); Hindi route (EasyOCR, self-detecting contract); padding and warm-up	bdcfbaa, a1d61d2, bb47ce0
9 Oct 2026	V2: PaddleOCR rewrite; V1 moved to legacy_v1/	e90636d
9 Oct 2026	Automatic script identification	d8f8290
9 Oct 2026	Prefer the Indic engine on mixed-script images; reading-order sorting	ec77275
9 Oct 2026	Documentation update for reading order and script detection	d8226b7
9 Oct 2026	Web interface overhaul and automated UI testing	713726c

6.2 Task Breakdown

Table 6.2 groups the work into tasks and records their status at the time of writing. Tasks that were planned but not carried out are listed so that the record of the project is complete.

Table 6.2 Task Breakdown and Status

Task	Outputs	Status
Audit and repair of the prototype	Grouping fix, box clamping, error handling, 21 legacy tests	Done
Configuration and registry	config.yaml loader, route registry, CLI flags	Done
Tamil and Hindi in V1	ocr_tamil and EasyOCR routes, padding sweep, synthetic pilots	Done
V2 re-architecture	PaddleOcrRecognizer, unified pipeline, reading-order sorting, 20 production tests	Done
Real-data evaluation	CER reports for 3 x 30 images, leakage analysis	Done
Script identification	script_detector module with Indic override, auto mode, UI chip	Done
Script identification evaluation	150-image run with design and held-out splits, both rules (Section 8.6)	Done

Task	Outputs	Status
Web application	FastAPI backend; redesigned single-file UI with Simple, Developer and Research views; 32 Playwright checks	Done
Comparison with other OCR tools	Feature comparison (Section 8.8)	Features: done. Accuracy: not run
Documentation	README, architecture, protocol, limitations, reproducibility	Done
Productionisation	Authentication, upload limits, GPU path, batching	Not started (future work)

6.3 Corrections Made During Documentation

Several statements were corrected when the evidence was re-examined during documentation. They are recorded here because they changed the claims of the report.

1. The English engine is PP-OCRv6, not PP-OCRv5. The code, tests and documents were corrected, and the vendor English accuracy figure, which belongs to a PP-OCRv5 model, is reported as context only.
2. An early “script-difficulty ordering” of English, Hindi and Tamil was withdrawn because English was measured on lines and the other two on words.
3. Latin-letter script leakage was discovered in the per-sample outputs and quantified.
4. “Synthetic data overestimates real accuracy” was restated as an evaluation-practice caution, because both the engine and the data changed between the two measurements.
5. The script identification evaluation runner was made incremental and resumable after an interrupted process lost about an hour of computation.
6. The statement that the script rule was frozen before evaluation with no fitted parameters was corrected: only the first (plain argmax) rule was fixed in advance; the override and its hand-chosen 25% threshold were added after failures were observed, so the final rule's held-out figure is indicative only.
7. The winner's share was removed from the interface as a confidence indicator because its median over 150 images is 1.0.

CHAPTER 7 — PROJECT DEMONSTRATION

7.1 Installation

The system is installed into a Python virtual environment from the public repository [26]. The first run of each language downloads its PaddleOCR weights into the user's PaddleX model cache.

```
git clone https://github.com/ripuns/Multilingual-OCR-Detection.git
cd Multilingual-OCR-Detection
python -m venv ocr_env
ocr_env\Scripts\activate          # Linux/macOS: source ocr_env/bin/activate
pip install -r requirements.txt        # production
pip install -r requirements-dev.txt    # pytest, datasets
pip install -r requirements-webapp.txt
```

7.2 Command-Line Usage

Without arguments the CLI runs automatic script identification on the sample page. A language can be forced with --script, which runs one engine instead of three and is therefore faster. Results are written to output/ocr_results.json and output/ocr_results.txt.

```
python main.py                                          # auto-detect on input/images/sample.png
python main.py --script tamil --input path\to\img.png  # force one language
python main.py --input img.png --output-dir out
```

7.3 Sample Run on an English Examination Page

Fig. 7.1 shows the sample page with the 16 regions that V2 returned, drawn from the stored output file of the repository. Boxes in blue have a confidence of at least 0.85; the single orange box is a stray “1” in the margin read with a confidence of 0.55. Each region is a complete line, for example Smoke sensor connected to External Interrupt 0 (INT0), which V1 had cut at “0 (”.

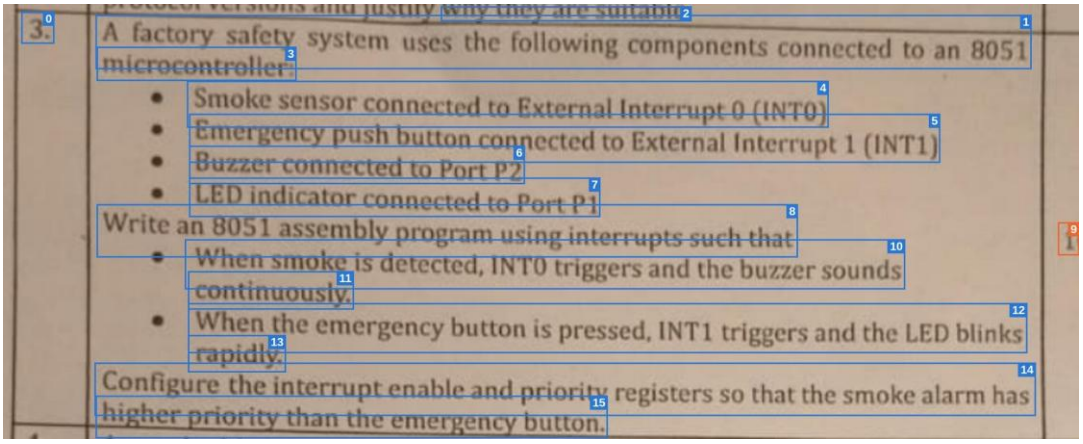


Fig. 7.1 Regions Returned by V2 on the Sample Examination Page

Table 7.1 Excerpt of the V2 Output for the Sample Page

Index	Recognised text	Score
1	A factory safety system uses the following components connected to an 8051	0.9914
3	microcontroller:	0.9999
4	Smoke sensor connected to External Interrupt 0 (INT0)	0.9912
5	Emergency push button connected to External Interrupt 1 (INT1)	0.9939
6	Buzzer connected to Port P2	1.0000
9	1	0.5502
12	When the emergency button is pressed, INT1 triggers and the LED blinks	0.9915

7.4 Web Demonstration

The web application is started with uvicorn webapp.app:app --host 127.0.0.1 --port 8000 and opened at http://127.0.0.1:8000. The status pill reads “Models ready” once the three engines are warm. Figs. 7.2 and 7.3 are

screenshots of the running application with the real models, showing an automatic run on the sample page. The Simple view (Fig. 7.2) presents the detected script, the combined text with Copy, Download and Share, a note about low-confidence lines, and buttons to re-read the image in another language. The Developer view (Fig. 7.3) adds the region boxes drawn over the image, the per-script evidence bars, and a table of regions with crops, engine labels and confidence pills. A Research tab, not reproduced here, renders the findings of Chapter 8 with their caveats. The interface was verified with a mock API driven by Playwright (32 checks, outside pytest) at 1320 px and 390 px widths, and against the real server.

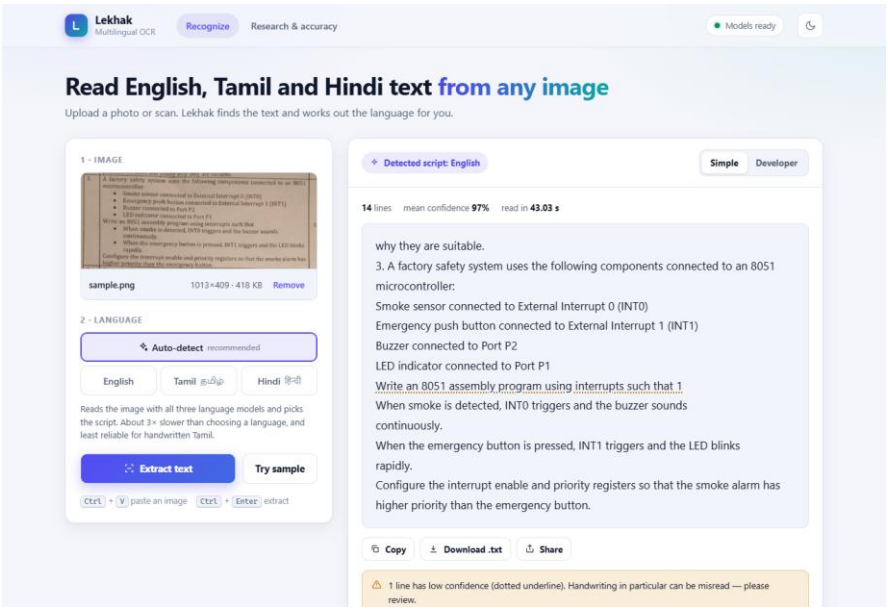


Fig. 7.2 Web Interface, Simple View: Auto-Detect Result on the Sample Page

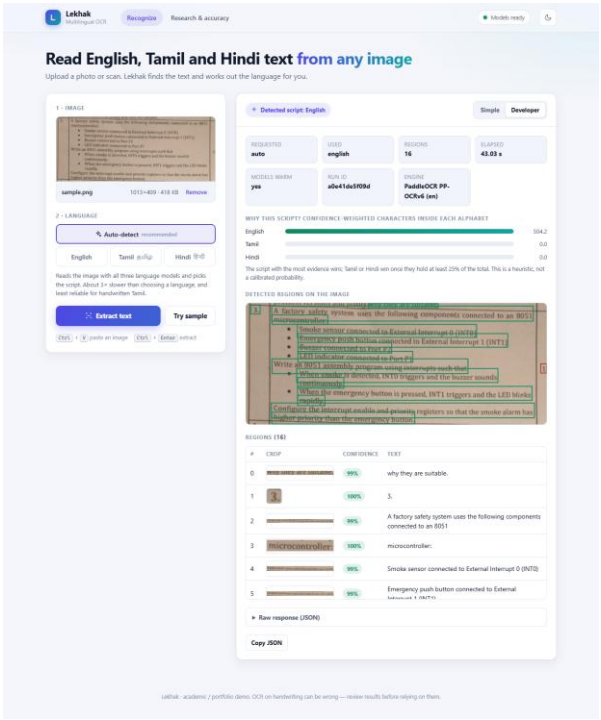


Fig. 7.3 Web Interface, Developer View: Region Boxes, Evidence Bars, Crops and Confidence

7.5 Testing

The command pytest tests/ legacy_v1/tests/ -q runs 41 tests (20 production, 21 legacy), all passing, without model downloads or network access. Table 7.2 lists them. File and crop writing in run_pipeline and the FastAPI endpoints are

not unit-tested; they were verified by live requests covering all three languages, crops, the health and research endpoints, an invalid script value, a path-traversal attempt and the frontend load. The 32 Playwright checks of the interface are separate from the 41 tests.

Table 7.2 Automated Test Inventory

Test file	Tests	Covers
tests/test_registry.py	4	Register and look up, missing route, three scripts registered, exact engine labels
tests/test_paddle_recognizer.py	3	Lazy construction (model not loaded); invalid script raises ValueError; reading-order sorting of lines and words
tests/test_script_detector.py	9	Latin, Tamil and Devanagari glyph counting; confidence weighting; argmax; out-of-block garbage ignored; Indic override on mixed-script images; English kept when the Indic share is small; no-evidence and empty cases
tests/test_main_recognize.py	4	Orchestration with fake recognisers: auto picks the right script and reuses its regions; English fallback flagged; manual runs only the chosen engine; masses and share reported
legacy_v1/tests/test_grouping.py	8	Grouping regression and adversarial cases, including left-drift cases that fail on pre-fix code
legacy_v1/tests/test_boxes.py	12	clamp_box and pad_and_clamp_box
legacy_v1/tests/test_east_detector.py	1	Missing image raises FileNotFoundError
Total	**41**	**20 production + 21 legacy; all passing**

CHAPTER 8 — RESULT AND DISCUSSION

8.1 Evaluation Methodology

Recognition accuracy is measured with the character error rate. For N evaluated images with predictions p_j and ground truths g_j, the overall CER is the sum of the Levenshtein distances divided by the sum of the reference lengths, as in Eqn. 8.1.

$$CER = \Sigma_j\, d(p_j, g_j) \,/\, \Sigma_j\, len(g_j), \quad j = 1 \ldots N \qquad (8.1)$$

In Eqn. 8.1, d is the minimum number of single-character insertions, deletions and substitutions, and len(g) is the length of the reference counted in Unicode code points. No Unicode or whitespace normalisation is applied, so a visible Tamil or Devanagari character spanning several code points can cost several edits. Exact match is string equality. Word error rate was not computed. Script identification accuracy is the fraction of images whose selected script equals the dataset's script. Table 8.1 lists the experiments.

Table 8.1 Experiments

ID	Experiment	Data	Output
E1	V1 grouping-bug ablation (qualitative text difference)	sample.png	Table 8.2
E2	EAST on Devanagari: box count at two thresholds	1 synthetic image	Section 8.2
E3	Tamil box-padding sweep (V1)	5 synthetic	Fig. 8.1
E4	Hindi pilot (V1, EasyOCR)	5 synthetic	Section 8.2
E5	V2 real-data CER	30 real x 3	Table 8.3
E6	Script-leakage analysis	E5 outputs	Table 8.5
E7	Script identification: plain argmax and argmax with Indic override, design and held-out splits	150 real	Tables 8.6–8.8
E8	Latency	150 real images; sample page	Table 8.9
E9	V1 vs V2 qualitative comparison	sample.png	Section 8.3

The following were controlled: a single machine, fixed model versions, deterministic inference, a fixed dataset order, identical CER code for all languages, and cached engine outputs so that both script rules are scored on exactly the same data. The following were not done: random sampling, repeated runs, confidence intervals, significance tests, grapheme-level CER, WER, comparison with other OCR engines, and running V1 on the real datasets.

8.2 V1 Results

Table 8.2 shows examples of the English grouping fix on the sample page (E1). Nineteen of 32 lines changed and all of them gained text. MICROCONTROLLE and a cut-off parenthetical were unchanged, because their cause was different.

Table 8.2 Examples of Text Recovered by the Grouping Fix

Before the fix	After the fix
SUITABLE.	JUSTLY WAY THEY ARE SUITABLE.
SAFETY SYSTEM	FACTORY SAFETY SYSTEM
8051	TO AN 8051
(INT1)	INTERRUPT 1 (INT1)
P2	ITO PORT P2
BUTTON	EMERGENCY BUTTON

Fig. 8.1 shows the Tamil padding sweep (E3), computed in-process over all regions of each image. The CER falls from 0.500 without padding to about 0.30 between 18 and 26 pixels and rises to 0.370 at 30 pixels. Through the real pipeline, scoring the first region only, a padding of 20 pixels gave a CER of 25.9% with one exact match in five: வணக்கம் was exact, while இது ஒரு சோதனை was still a severe failure (CER 0.77). The absolute values of the two procedures differ slightly; the trend is the finding, from five images in one font.

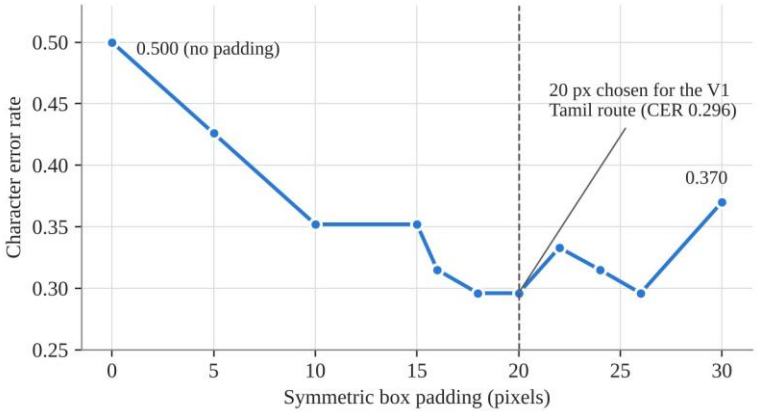


Fig. 8.1 Tamil CER Against Box Padding in V1 (5 Synthetic Images)

For Hindi (E4), EasyOCR achieved a CER of 7.1% with three exact matches in five synthetic images. For E2, EAST returned zero boxes on the synthetic Devanagari image at confidence thresholds of 0.3 and 0.05, while EasyOCR's detector read the same image with a confidence of 0.90.

8.3 V2 Qualitative Results on the Sample Page

On the same sample page V2 returned 16 coherent regions where V1 had returned 32 fragments (E9). Smoke sensor connected to External Interrupt 0 (INT0) is complete, whereas V1 cut it at “0 (”; microcontroller: is read correctly, where V1 produced MICROCONTROLLE; and Emergency push button connected to External Interrupt 1 (INT1) is read as a single line. Confidences were mostly between 0.98 and 1.0; the one low value was the stray “1” at 0.55, as Fig. 7.1 showed.

8.4 V2 Real-Data Character Error Rate

Table 8.3 gives the results of E5. The English engine was scored on handwritten sentence lines and the Tamil and Hindi engines on isolated handwritten words, from different datasets, writers and capture conditions.

Table 8.3 V2 Character Error Rate on Real Handwriting (30 Images per Language)

Script	Dataset	Unit	Mean GT chars	CER	Exact	Median CER	CER ≤ 0.25	CER ≥ 1.0	Empty
English	IAM-line	line	60.9	24.1%	0/30	0.146	23	2	2
Hindi	IIIT-INDIC-HW-WORDS-Hindi	word	6.4	47.7%	2/30	0.333	12	8	0
Tamil	IIIT-INDIC-HW-WORDS-Tamil	word	9.6	71.1%	1/30	0.909	4	12	0

Fig. 8.2 plots these values beside the V1 synthetic pilot values. Fig. 8.3 shows how the thirty samples of each language are distributed between good transcriptions (CER at most 0.25), partial errors and complete failures (CER of 1.0 or more). Tamil has the most complete failures and English the most good transcriptions, but because the units differ these numbers must not be read as a ranking of script difficulty.

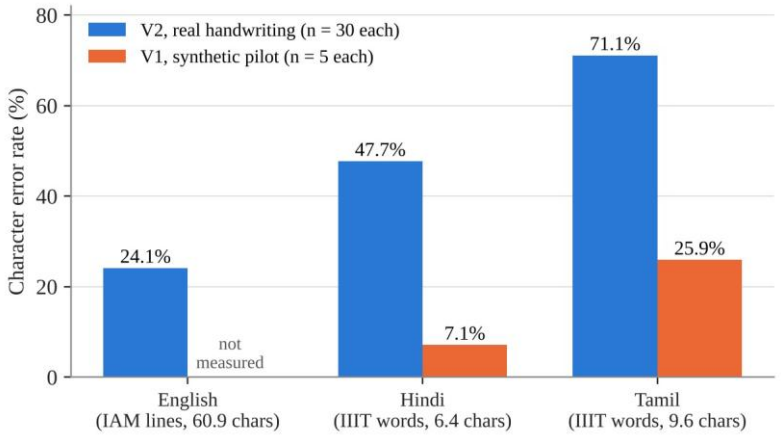


Fig. 8.2 V2 Real-Data CER With V1 Synthetic Pilot Values (Units Differ Across Languages)

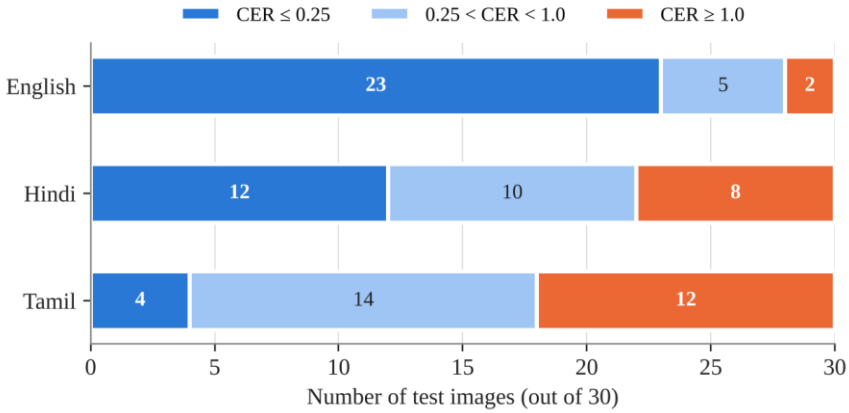


Fig. 8.3 Distribution of Per-Image CER for Each Language

Table 8.4 shows representative examples from the stored per-sample outputs. Good English lines differ from the reference by a single character, while two English lines produced no output at all. Hindi errors range from single substitutions to Latin garbage, and the worst Tamil errors are entirely in Latin letters.

Table 8.4 Examples of Ground Truth, Prediction and Per-Sample CER

Script	Ground truth	Prediction	CER
English	He is also a director of a couple of garages . And he finds time as well to be a lyric	Same text, but “garages.” without the space	0.012
English	the switch because of the topicality of African	the swith because of the topicality of African	0.021
English	She plays the possessive mother of a man whose hobby revolves	(empty)	1.000
Hindi	टाटा	टाटा (exact)	0.000
Hindi	केंद्रों	केंदट्रों	0.125
Hindi	डीजल	Dutnt	1.250
Tamil	எடுக்கலாம்	எடுக்கலாம் (exact)	0.000
Tamil	இந்தியர்களுக்கு	இந்தியர்கமுக்கு	0.067
Tamil	பேச்சைக்	BODA	1.000
Tamil	ஏளனம்	Jaostlo	1.400

8.5 Script Leakage

Inspection of the per-sample outputs (E6) revealed that the Indic engines often output Latin letters when they fail. Table 8.5 and Fig. 8.4 quantify this. The Tamil engine produced Latin letters in 17 of its 30 predictions, and those predictions had a CER of 0.912 against 0.461 for the rest, of which nine were entirely in Tamil script. The Hindi engine did so in only

two predictions. Examples from the Tamil engine include nigeggn, uoog, BODA, Bnonbe, TளியவT and aாmi; from the Hindi engine, tt and Dutnt. The English engine instead returned empty strings for two of its thirty lines.

Table 8.5 Script Leakage in the Indic Engines

Engine	Non-empty predictions	With Latin letters	CER on those	CER on the rest
Tamil	30	17 (57%)	0.912	0.461 (13; 9 fully in Tamil)
Hindi	30	2	1.167	0.455 (28; 25 fully in Devanagari)

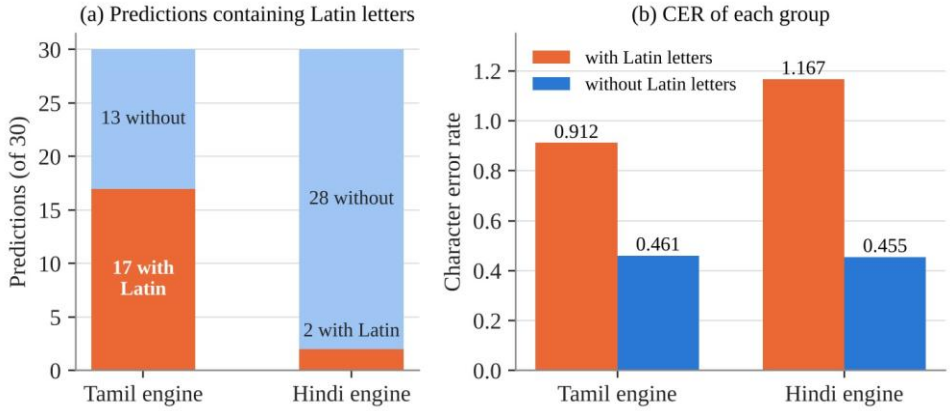


Fig. 8.4 Latin Script Leakage: Frequency and Effect on CER

The likely explanation, inferred from the outputs rather than from the model dictionaries, is that the multilingual recognisers include Latin characters in their output vocabulary and fall back to them on hard input. Leakage matters twice: it inflates the CER, and it weakens script identification, because Latin letters emitted by the Tamil engine add nothing to Tamil's glyph mass.

8.6 Automatic Script Identification

E7 evaluates the script rules of Section 4.1.10 on 150 real images, the first 50 of each of the three datasets. Images 0 to 29 of each language are byte-identical to the CER images and were seen while the first rule was designed (the design split); images 30 to 49 were not examined beforehand (the held-out split). Two rules were scored on the same cached engine outputs. The **first rule** (plain argmax) was fixed before any of this data was scored, so its held-out figure is a clean estimate. The **final rule** (argmax with the Indic override at 25%) was introduced after failures were observed and before this evaluation ran, with one hand-chosen threshold, so its held-out figure is indicative and not a clean unseen-data estimate. The runner stores every engine output in raw_outputs.jsonl so that the slow three-engine run can resume after interruption, and a separate script computes accuracy and the confusion matrix.

Table 8.6 Script Identification Accuracy of the Final Rule (Argmax With Indic Override), E7

Split	Images	English correct	Tamil correct	Hindi correct	Overall accuracy
Design (0–29)	90	28/30	15/30	26/30	76.7% (69/90)
Held-out (30–49)	60	20/20	10/20	17/20	78.3% (47/60), indicative
All	**150**	**48/50**	**25/50**	**43/50**	**77.3% (116/150)**

Table 8.7 Script Identification Accuracy of the First Rule (Plain Argmax), E7

Split	Images	English correct	Tamil correct	Hindi correct	Overall accuracy
Design (0–29)	90	28/30	10/30	18/30	62.2% (56/90)
Held-out (30–49)	60	20/20	8/20	14/20	70.0% (42/60), clean estimate
All	**150**	**48/50**	**18/50**	**32/50**	**65.3% (98/150)**

Table 8.8 Confusion Matrix of the Final Rule (All 150 Images; Rows Are the True Script, Columns the Prediction)

True \ predicted	english	tamil	hindi	none
english	48	0	0	2
tamil	18	25	2	5
hindi	6	0	43	1

“None” means no engine produced any in-script text, in which case the pipeline returns English with fallback set to true. On the held-out split alone the final rule was correct on English 20/20, Hindi 17/20 (three read as English) and Tamil 10/20 (six read as English, four none).

- **English is reliable (96%).** Both misses are images on which the English engine itself returned an empty string.
- **The override helped.** It raised overall accuracy from 65.3% to 77.3% (Hindi 64% to 86%, Tamil 36% to 50%) and misrouted none of the 50 English images to an Indic script. This was measured on the same 150 images the override was designed around, so it is indicative.
- **Tamil is the weak case (50%), and the cause is the recogniser, not the scoring rule.** Of the 25 misidentified Tamil images, in 21 the Tamil engine's output contained no Tamil characters at all, and in 3 the Tamil share was below 25%. Of the 7 misidentified Hindi images, 5 had no Devanagari at all. When the matching engine cannot read the script, no rule built on engine outputs can recover it; this is the leakage of Section 8.5 seen from the identification side.
- **The winner's share is not a usable confidence.** Its median is 1.0 for every split, so the interface does not present it as one.

8.7 Latency

Table 8.9 reports timings on the development CPU with warm models. The per-engine figures are medians over the 150 images of E7; two Tamil images (tamil_5.png and tamil_8.png) took 2,178.9 s and 10,693.3 s because the laptop was suspended during the run, which is why medians and not means are reported. V2 processed the sample page in 14.5 seconds in manual English mode against 126.8 seconds for V1, whose TrOCR-large recogniser took about four seconds per region; the same page in automatic mode took 39.5 s. Automatic mode costs roughly the sum of the three engine times, consistent with the 13.27 s median against 2.95–5.16 s for a single engine. All timings are single-machine measurements and were not replicated.

Table 8.9 Latency on the Development CPU (Warm Models)

Case	Time
English engine, per image (median of 150)	2.95 s
Tamil engine, per image (median of 150)	5.16 s
Hindi engine, per image (median of 150)	5.11 s
All three engines, automatic mode, per image (median of 150)	13.27 s
V2 automatic mode, English sample page (16 regions)	39.5 s
V2 English, sample page, 16 regions, manual (single run)	14.5 s
V2 Tamil, one real word image, manual (single run)	6.4 s
V2 Hindi, one real word image, manual (single run)	2.5 s
V1 English, sample page, 32 regions (TrOCR-large, about 4 s per region)	126.8 s
V1 Tamil or Hindi, one-region synthetic image	about 0.4 s

8.8 Feature Comparison With Other OCR Options

No accuracy comparison was run against any other tool in this project, and none is claimed. Table 8.10 compares features and deployment properties only, compiled from general knowledge of public vendor documentation; cells marked “check vendor docs” were not verified and should be confirmed against the vendors' current documentation before the report is relied on.

Table 8.10 Feature Comparison With Other OCR Options (Features Only; No Accuracy Comparison Was Run)

Tool	Local / offline	Cost	English + Tamil + Hindi script handling	Per-region confidence	Open source	Real-handwriting accuracy measured here
This system (V2)	Yes	No per-image fee	Automatic choice among three recognisers; one script per image	Yes: region score, crop and engine label	Yes (Apache 2.0 PaddleOCR stack)	24.1% / 47.7% / 71.1% CER (different units, not comparable)
Tesseract	Yes	No per-image fee	Caller selects language data; an OSD mode detects the script	Yes (word level)	Yes	Not measured
EasyOCR	Yes	No per-image fee	Caller passes a language list; Tamil and Hindi models exist	Yes	Yes	Not measured
PaddleOCR used directly	Yes	No per-image fee	One language per call; no automatic choice	Yes (per-line score)	Yes (Apache 2.0)	Same engines as V2; not measured separately
Google Cloud Vision OCR	No (cloud API)	Usage-based; check vendor docs	Detects language automatically; check vendor docs for Tamil and Hindi handwriting	Yes (word / symbol level)	No	Not measured
Microsoft Azure AI Vision Read	No (cloud API)	Usage-based; check vendor docs	Check vendor docs	Yes (word level)	No	Not measured
Amazon Textract	No (cloud API)	Usage-based; check vendor docs	Check vendor docs	Yes (per block)	No	Not measured

The differentiators of this system are therefore operational and evidential rather than accuracy-based: it runs fully locally with no per-image cost and no image upload; it chooses among English, Tamil and Hindi by comparing the outputs of three recognisers; it returns a crop, a confidence and an engine label for every region; and it reports its own measured accuracy on real third-party handwriting together with its failure modes, backed by reproducible evaluation scripts and 41 automated tests. The trade-offs are equally real: automatic mode costs about three times the compute (13.27 s median per image on this CPU), only one script is assigned per image, Tamil identification is only 50% on real handwriting, and handwriting CER is high. Automatic script detection is not unique to this system, since Tesseract also offers script detection. Cloud services may well be faster or more accurate, but that was not measured here. The system should be read as a different trade-off, local, transparent and honestly evaluated, not as a better recogniser.

8.9 Vendor-Reported Accuracy

For context, PaddleOCR publishes recognition accuracies of 85.25% for en_PP-OCRv5_mobile_rec, 94.2% for ta_PP-OCRv5_mobile_rec and 84.96% for devanagari_PP-OCRv5_mobile_rec, each measured on the vendor's own test sets. The English figure belongs to a PP-OCRv5 model, whereas this project's English pipeline uses PP-OCRv6. These figures are not comparable with the real-handwriting results above, and the large gap, particularly for Tamil, was not diagnosed.

8.10 Discussion

Architecture. Moving from a per-language patchwork to one engine per language removed an entire class of failures, namely detector incompatibility and grouping, and collapsed three code paths into one. The price is dependence on PaddleOCR's internal model selection and a heavier engine than a simple detector.

Accuracy on real handwriting is the main weakness. Even with a modern engine, Tamil word recognition is mostly wrong, with a median CER of 0.91; Hindi is mixed, with a median of 0.33; and English sentence lines are comparatively good, with a median of 0.15. Because the units and datasets differ, these numbers describe the system on these data, not the relative difficulty of the scripts.

Synthetic and real data. The V1 synthetic pilots suggested CERs of 7.1% for Hindi and 25.9% for Tamil, while real handwriting gave 47.7% and 71.1% on V2. Since both the engine and the data changed, the safe lesson concerns practice: validate on a real third-party dataset before quoting accuracy. Informally, V2 also read the clean synthetic Hindi images well, which suggests that data difficulty rather than the engine change drives the gap, but this was not saved as a result.

Convenience against compute. Automatic identification removes a decision the user may be unable to make, at about three times the compute of manual mode, and its accuracy is bounded by the recognisers' own failure modes: 96% for English and 86% for Hindi, but 50% for Tamil. Manual mode remains the fast path, and the only reliable one for Tamil, when the language is known; the interface offers “Read as” re-runs for that reason.

8.11 Limitations and Threats to Validity

1. Samples are small (30 per language for CER, 150 for script identification), taken from the first rows of each dataset, and reported without confidence intervals.
2. English lines and Tamil and Hindi words come from different datasets, so CER is not comparable across languages.
3. Code-point CER over-penalises Indic graphemes, and WER was not computed.
4. The V1 synthetic and V2 real comparison mixes an engine change with a data change, and V1 was never evaluated on the real datasets.
5. Script leakage and empty outputs inflate the error and are not detected or corrected by the system.
6. Only the first script rule was fixed before the data were scored; the final rule's override and its hand-chosen 25% threshold were introduced after failures were observed, so its held-out figure (78.3%) is indicative, and only the first rule's held-out figure (70.0%) is a clean estimate. The design split also overlaps the CER images.
7. Automatic mode costs about three times the compute, assigns one script per image and supports three scripts only.
8. The EAST failure on Devanagari was shown on one synthetic image and one font.
9. The web application has no authentication, rate limiting or cleanup, CORS is open, and the 15 MB upload limit is enforced in the browser only. It was tested in Edge (Chromium) at 1320 px and 390 px, not in Firefox or Safari, and was not audited with a screen reader.
10. Latency figures are from one CPU-only laptop and have not been replicated; the dataset and EAST-weight licences require review.
11. The feature comparison with other tools was compiled without querying vendor documentation and contains unverified cells; no other tool was run.

CHAPTER 9 — SUMMARY

This project set out to build a multilingual OCR tool for English, Tamil and Hindi that does not ask the user to name the language and that is honest about its accuracy. The delivered system runs one PaddleOCR detection-and-recognition engine per language, identifies the script automatically from the confidence-weighted in-script glyph mass of the three engines' outputs with an override for Indic scripts, and is available through a command-line interface and a FastAPI web application with Simple, Developer and Research views. It is covered by 41 automated tests that need neither models nor network access.

The path to this design is itself a result. The modular Review-1 pipeline contained a grouping defect that silently clipped text; once fixed, the same pipeline failed differently for each new script, producing under-sized boxes for Tamil and no boxes at all for Hindi. Replacing it with a single engine family removed these failures, reduced the sample-page latency from 126.8 to 14.5 seconds, and replaced 32 fragments with 16 coherent lines.

On real handwriting the system remains far from production quality: 24.1% CER on English sentence lines, 47.7% on Hindi words and 71.1% on Tamil words. A large part of the Tamil error comes from script leakage, in which the Tamil engine emits Latin letters on 17 of 30 images. Automatic script identification was correct on 77.3% of 150 real images (English 96%, Hindi 86%, Tamil 50%), against 65.3% for the first rule; the Tamil failures are recogniser failures, since in 21 of 25 the Tamil engine produced no Tamil characters. A clean synthetic pilot had suggested much better accuracy, which is the strongest practical lesson of the project: accuracy must be measured on real third-party data before it is quoted.

Recommendations and Future Work

- Improve the Tamil recogniser path, or fall back to a different Tamil engine, since 21 of the 25 Tamil identification failures are recogniser failures.
- Detect and suppress script leakage, for example by rejecting Latin output from an Indic engine or re-running the correct engine on low-confidence regions.
- Identify the script per region rather than per image so that documents mixing scripts can be read.
- Evaluate on larger, randomly sampled, word-level data for all three languages with grapheme-level CER, WER and confidence intervals, and run V1 and other engines, including the tools of Table 8.10, on the same data.
- Investigate the gap between the vendor-reported Tamil accuracy and the real-handwriting result.
- Add further scripts, which requires one language-code entry, one engine label and an evaluation, with low-resource scripts as the most open research direction.
- Prepare the web service for deployment with authentication, server-side upload limits, cleanup, asynchronous processing, batching and a GPU path.

The contribution of the project is therefore a working, tested and documented multilingual OCR system together with a clear, evidence-backed account of where it works, where it fails, and why.

CHAPTER 10 — REFERENCES

[1] Smith, R., Antonova, D., & Lee, D.-S. (2009). Adapting the Tesseract open source OCR engine for multilingual OCR. In *Proceedings of the International Workshop on Multilingual OCR (MOCR '09)*, Article 1. Association for Computing Machinery. https://doi.org/10.1145/1577802.1577804

[2] Zhou, X., Yao, C., Wen, H., Wang, Y., Zhou, S., He, W., & Liang, J. (2017). EAST: An efficient and accurate scene text detector. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 5551–5560). IEEE. https://doi.org/10.1109/CVPR.2017.283

[3] Shi, B., Bai, X., & Yao, C. (2017). An end-to-end trainable neural network for image-based sequence recognition and its application to scene text recognition. *IEEE Transactions on Pattern Analysis and Machine Intelligence, 39*(11), 2298–2304. https://doi.org/10.1109/TPAMI.2016.2646371

[4] Baek, Y., Lee, B., Han, D., Yun, S., & Lee, H. (2019). Character region awareness for text detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 9365–9374). IEEE.

[5] Li, M., Lv, T., Chen, J., Cui, L., Lu, Y., Florencio, D., Zhang, C., Li, Z., & Wei, F. (2023). TrOCR: Transformer-based optical character recognition with pre-trained models. *Proceedings of the AAAI Conference on Artificial Intelligence, 37*(11), 13094–13102. https://doi.org/10.1609/aaai.v37i11.26538

[6] Bautista, D., & Atienza, R. (2022). Scene text recognition with permuted autoregressive sequence models. In *Computer Vision – ECCV 2022* (Lecture Notes in Computer Science). Springer.

[7] Du, Y., Li, C., Guo, R., Yin, X., Liu, W., Zhou, J., Bai, Y., Yu, Z., Yang, Y., Dang, Q., & Wang, H. (2020). *PP-OCR: A practical ultra lightweight OCR system* (arXiv:2009.09941). arXiv. https://doi.org/10.48550/arXiv.2009.09941

[8] Cui, C., Sun, T., Lin, M., Gao, T., Zhang, Y., Liu, J., Wang, X., Zhang, Z., Zhou, C., Liu, H., Zhang, Y., Lv, W., Huang, K., Zhang, Y., Zhang, J., Zhang, J., Liu, Y., Yu, D., & Ma, Y. (2025). *PaddleOCR 3.0 technical report* (arXiv:2507.05595). arXiv. https://doi.org/10.48550/arXiv.2507.05595

[9] Zhang, Y., Wang, X., Lin, M., Zhang, Y., Deng, P., Sun, T., Gao, T., Zhang, Z., Liu, J., Zhou, C., Liu, H., Liang, S., Cui, C., Liu, Y., Yu, D., & Ma, Y. (2026). *PP-OCRv6: From 1.5M to 34.5M parameters, surpassing billion-scale VLMs on OCR tasks* (arXiv:2606.13108). arXiv. https://doi.org/10.48550/arXiv.2606.13108

[10] Marti, U.-V., & Bunke, H. (2002). The IAM-database: An English sentence database for offline handwriting recognition. *International Journal on Document Analysis and Recognition, 5*(1), 39–46. https://doi.org/10.1007/s100320200071

[11] Gongidi, S., & Jawahar, C. V. (2021). IIIT-INDIC-HW-WORDS: A dataset for Indic handwritten text recognition. In *Document Analysis and Recognition – ICDAR 2021* (Lecture Notes in Computer Science). Springer.

[12] Shi, B., Bai, X., & Yao, C. (2016). Script identification in the wild via discriminative convolutional neural network. *Pattern Recognition, 52*, 448–458. https://doi.org/10.1016/j.patcog.2015.11.005

[13] Ubul, K., Tursun, G., Aysa, A., Impedovo, D., Pirlo, G., & Yibulayin, T. (2017). Script identification of multi-script documents: A survey. *IEEE Access, 5*, 6546–6559.

[14] Gupta, A., Vedaldi, A., & Zisserman, A. (2016). Synthetic data for text localisation in natural images. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 2315–2324). IEEE. https://doi.org/10.1109/CVPR.2016.254

[15] Yang, Z., Tang, J., Li, Z., Wang, P., Wan, J., Zhong, H., Liu, X., Yang, M., Wang, P., Bai, S., Jin, L., & Lin, J. (2025). CC-OCR: A comprehensive and challenging OCR benchmark for evaluating large multimodal models in literacy. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)* (pp. 21744–21754). IEEE. https://doi.org/10.48550/arXiv.2412.02210

[16] Levenshtein, V. I. (1966). Binary codes capable of correcting deletions, insertions, and reversals. *Soviet Physics Doklady, 10*(8), 707–710.

[17] Liao, M., Wan, Z., Yao, C., Chen, K., & Bai, X. (2020). Real-time scene text detection with differentiable binarization. *Proceedings of the AAAI Conference on Artificial Intelligence, 34*(7), 11474–11481. https://doi.org/10.1609/aaai.v34i07.6812

[18] Dutta, K., Krishnan, P., Mathew, M., & Jawahar, C. V. (2018). Offline handwriting recognition on Devanagari using a new benchmark dataset. In *13th IAPR International Workshop on Document Analysis Systems (DAS)* (pp. 25–30). IEEE.

[19] The Unicode Consortium. (2024). *The Unicode Standard* (Version 16.0.0). https://www.unicode.org/versions/Unicode16.0.0/

[20] PaddlePaddle. (2026). *PaddleOCR* (Version 3.7.0) [Computer software]. GitHub. https://github.com/PaddlePaddle/PaddleOCR

[21] JaidedAI. (2020). *EasyOCR* [Computer software]. GitHub. https://github.com/JaidedAI/EasyOCR

[22] GnanaPrasath. (2024). *ocr_tamil* [Computer software]. Python Package Index. https://pypi.org/project/ocr-tamil/

[23] Ramírez, S. (2018). *FastAPI* [Computer software]. https://fastapi.tiangolo.com/

[24] Teklia. (2023). *IAM-line* [Data set]. Hugging Face. https://huggingface.co/datasets/Teklia/IAM-line

[25] c3rl. (2024). *IIIT-INDIC-HW-WORDS-Tamil and IIIT-INDIC-HW-WORDS-Hindi* [Data sets]. Hugging Face. https://huggingface.co/datasets/c3rl/IIIT-INDIC-HW-WORDS-Tamil

[26] Sethia, R., & Joshi, Y. (2026). *Multilingual-OCR-Detection* [Source code]. GitHub. https://github.com/ripuns/Multilingual-OCR-Detection

APPENDIX A

This appendix contains selected source code and supporting material. Only representative code is included; the complete source is available in the project repository [26].

A.1 Script Identification Module (recognition/script_detector.py)

```
SCRIPT_BLOCKS = {
    "tamil": lambda ch: "\u0b80" <= ch <= "\u0bff",
    "hindi": lambda ch: "\u0900" <= ch <= "\u097f",
    "english": lambda ch: ("a" <= ch <= "z") or ("A" <= ch <= "Z"),
}
INDIC_SCRIPTS = ("tamil", "hindi")
INDIC_MIN_SHARE = 0.25

def script_mass(script, regions):
    """Confidence-weighted count of characters belonging to script's block."""
    in_block = SCRIPT_BLOCKS[script]
    return sum(
        r.get("score", 0.0) * sum(1 for ch in r["text"] if in_block(ch))
        for r in regions
    )

def pick_script(results_by_script):
    masses = {s: script_mass(s, regions) for s, regions in results_by_script.items()}
    total = sum(masses.values())
    if total <= 0:
        return None, {"masses": masses, "share": 0.0}
    best = max(masses, key=masses.get)
    indic = [s for s in INDIC_SCRIPTS if s in masses]
    if indic:
        best_indic = max(indic, key=masses.get)
        if masses[best_indic] / total >= INDIC_MIN_SHARE:
            best = best_indic
    return best, {"masses": masses, "share": masses[best] / total}
```

A.2 Orchestration of Manual and Automatic Modes (main.py)

```
def recognize(image_path, script):
    if script != "auto":
        return _get_recognizer(script).detect_and_recognize(image_path), {
            "mode": "manual", "script": script, "fallback": False,
            "share": None, "masses": None,
        }
    by_script = {s: _get_recognizer(s).detect_and_recognize(image_path) for s in SCRIPTS}
    picked, info = pick_script(by_script)
    chosen = picked or FALLBACK_SCRIPT
    return by_script[chosen], {
        "mode": "auto", "script": chosen, "fallback": picked is None,
        "share": round(info["share"], 4),
        "masses": {s: round(m, 3) for s, m in info["masses"].items()},
    }
```

A.3 Reproducing the Evaluation and Running the Demo

```
python experiments/runs/pull_datasets.py                  # real images from Hugging Face
python experiments/runs/run_paddleocr_eval.py all         # E5: CER reports
python experiments/runs/script_id/run_engines.py          # E7: 3 engines x 150 images (resumable)
python experiments/runs/script_id/evaluate.py             # E7: script-ID accuracy, both rules
pytest tests/ legacy_v1/tests/ -q                         # 41 tests
uvicorn webapp.app:app --host 127.0.0.1 --port 8000
python -m legacy_v1.main --script english                 # V1 (needs legacy requirements + EAST weights)
```